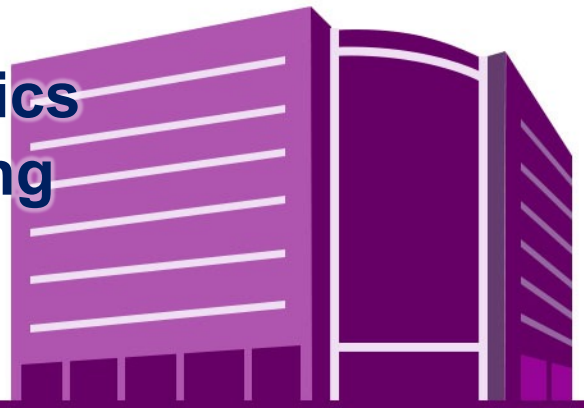
Fundamentals of Computer Programming (I)
—— C Programming Language

Course Introduction

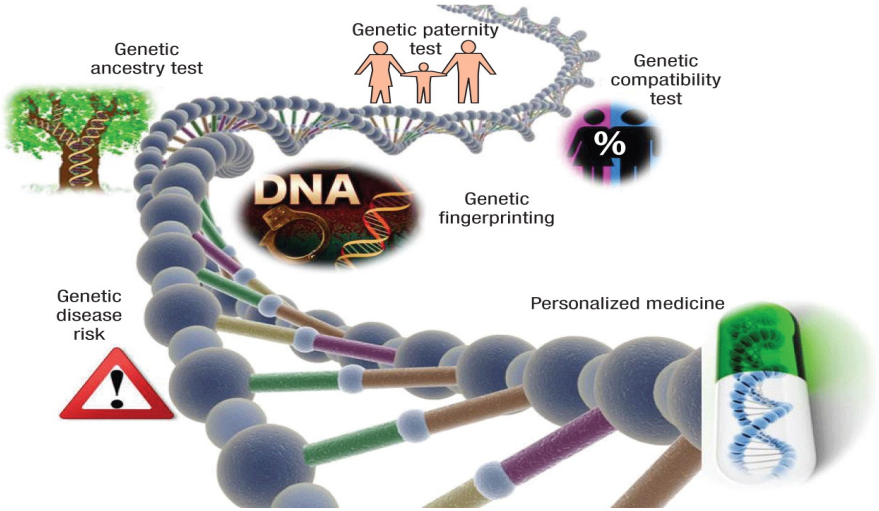
Xing Lin, Assistant Professor
Institute of Information Optoelectronics
Department of Electronic Engineering



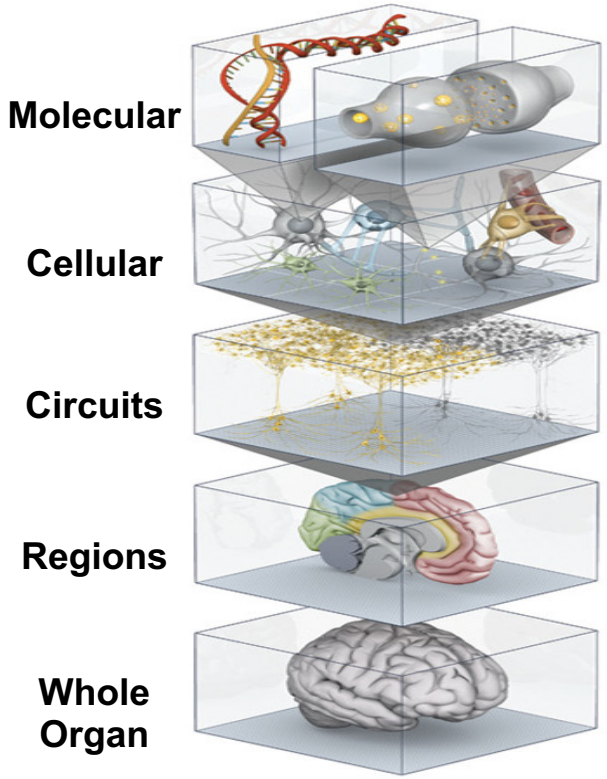
Office: Rohm Building 5-208, Tel: 010-6279-2127, Email: lin-x@tsinghua.edu.cn

Programming in nature

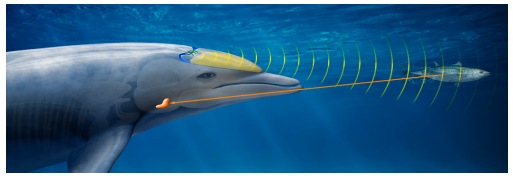
- **Two examples:** (1) One is the formation of living organisms via programming. Genes carry life information, composed of ATCG base pairs, which transcribe into proteins, forming life. (2) Learning process of animal brains. Neurons collaborate, generate pulse sequences, and execute programs to complete tasks.

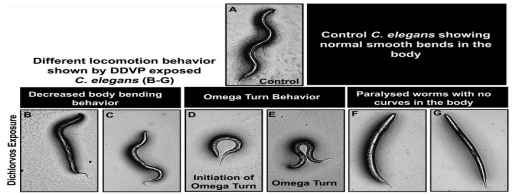


Genetics



Brain



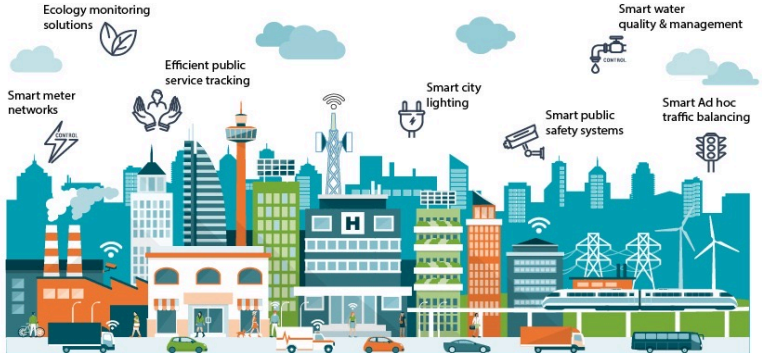


Animal Behaviors

Modern information life driven by software

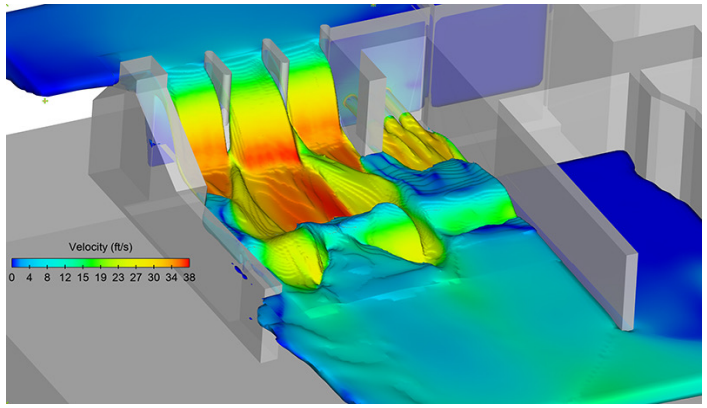
- **Programs are the cornerstone of the information society, driving systems like the following research areas that all rely on software support.**

Artificial Intelligence



Smart City

Operating System



Civil and Hydraulic Engineering

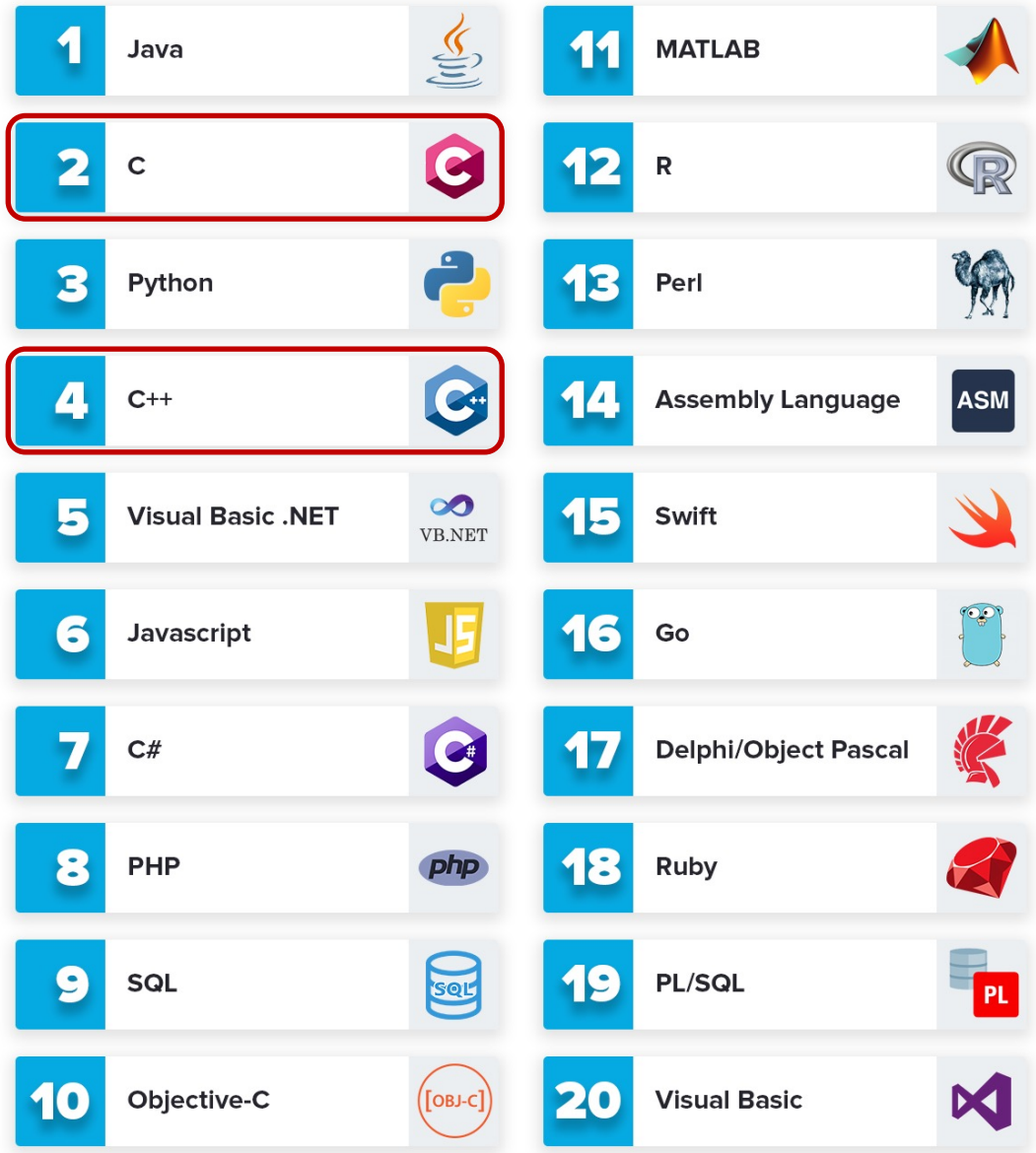
Biological & Chemical Engineering

Defense and Military

Programming languages

The advantages of C/C++ programming

- Portability across platforms
- Low-level hardware access
- Speed and efficiency with compilation
- Extensibility with rich libraries
- Procedural & object-oriented
- Widespread use and community support
- Suitability for operating system programming and hardware control
- Much Easier to learn other languages after learning C/C++ programming



Programming languages

The advantages of C/C++ programming

- With a long history and outstanding achievements
- Born in 70's, mature in 80's, revised in 90's
- A lot of heavyweight software is written in C/C++
- There is almost no software that cannot be written in C/C++, and no system that does not support C/C++
- Many popular languages and new languages have borrowed its ideas and syntax, e.g., from C++ to Java, C#, php, python and more

Ken Thompson & Dennis Ritchie

National Medal of Technology

Turing Award

Teaching objective

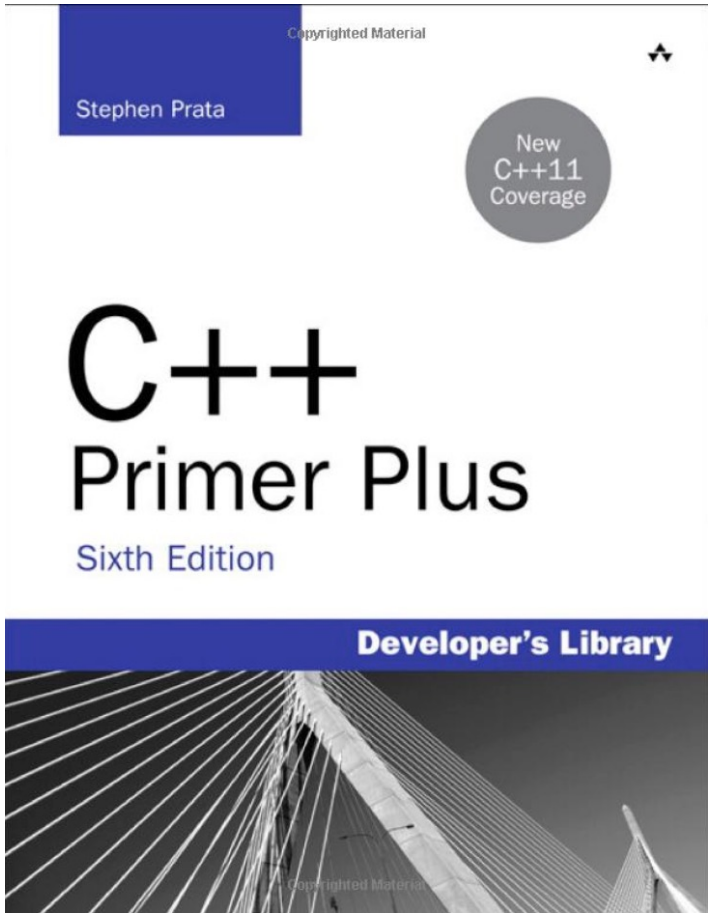
- Programming exercises thinking and cultivates logical ability
- Master the basic knowledge and methods of process-oriented C programming, including basic programming and debugging skills
- Develop good programming skills
  - Comments + alignment: emphasize code readability
  - Clearly documented: program ideas should be explained
  - Debug: learn to find program bugs
  - Optimization: algorithm and structure as best as possible
  - Test: correctness analysis of run results
- Focus on method learning and skill training
- Hands-on practice on the computer and encourage innovation

Teaching philosophy and methods

- **How to learn the course:** True knowledge comes from practice = Teacher impart wisdom (teach basic skills) + students practice and comprehend (diligently study, practice hard, ask questions)
- **How to practice:** self-study and problem solving + computer application + online communication
- **Challenges in practice:** self-reliance + self-confidence + self-improvement
- **Programming qualities:** interest + patience + endurance (craftsmanship spirit)
- **Evaluation criteria:** It's not only about "whether you understand", but about "whether you can write"; emphasizing practical ability
- **Learning attitude:** Don't overestimate your programming skills, step by step and don't hurry too much

Teaching materials and references

黄永峰，孙甲松编著；
《C/C++程序设计教程》，
清华大学出版社, 2019



Stephen Prata编著；
《C/C++ Primer Plus》，
人民邮电出版社 (英文版), 2022

Course schedule and assessment method

- 1st to 16th week, 2 class hour per week
- **Final score:** (1) Final written test (40%): read/write program; (2) Computer-based exam (30%): midterm (10%) and final (20%); (3) Weekly homework and experiments (30%)
- **Homework and computer-based experiments:** (1) Follow the requirements; guarantee both quality and quantity; the programming questions are required to be completed on the computer. (2) Submit the electronic version online before the deadline. If you miss the deadline, you can send an email to me.
- **Open discussion for plus: Tsinghua web learning** (please leave the contact information), **email**, and **open office hour** (Weekly Monday, 14:00-16:00, Rohm Building 3-105).

About Tsinghua web learning

- **Courseware and homework will be released on Tsinghua web learning after each class**
- **Reference material including the digital textbooks and reference answers to after-class exercises**
- **The contact information of two teaching assistants can be found on Tsinghua web learning**
- **Pay attention to the homework DDL and submission status, where the attachment size is 0 means the submission is not successful**
- **We encourage you to ask questions and participate in the discussion!**
  Teacher and three teaching assistants will answer them as soon as possible. It is better to put your source program and error results (screenshots, etc.) in a word file so that others can compile and run to reproduce your errors

About computer-based experiments

- From week 2 to week 15, there are five periods per week (will be announced). The computer-based experiments are with the classes of Prof. Sun, Prof. Huang, and Prof. Yang, choose one (second-level elective course), 4 hours each time.
- Location: Microcomputer Lab on the East Side of the 9th floor of the Central Main Building

- Study hard, practice harder!

About AI-assisted teaching and learning

- **This course supports artificial intelligence (AI) large language model (LLM) as an assistive teaching tool and learning partner.**
- **LLM platforms:** commercial LLM, AI plug-ins, and rain classroom
- **The advantages of LLM:** can give appropriate explanations, sample code, grammar check, and debugging suggestions for programming tasks, and can also provide personalized learning suggestions

Commercial LLM

文心一言 智谱清言 通义千问 Moonshot AI

AI Plug-ins

GitHub Copilot Intellicode Making Intellisense In Visual Studio

Rain Classroom (荷塘雨课堂)

https://pro.yuketang.cn/

学堂在线 xuetangx.com

Knowledge Graph

24h Learning Companion

About AI-assisted teaching and learning

- **Reminder of using LLM:**

(1) AI-assisted learning tools are there to help you, so don't be afraid to use them! You can absolutely refer to and use AI tools in your regular assignments. Just make sure to avoid unthinking reliance or simply copy-and-paste, which might not be the best for your learning experience and quality.

(2) In the exam and computer-based practice in microcomputer lab, you'll be disconnected from the internet to prevent AI help. This is just to make sure you're using your own skills and knowledge. You'll also be required to write code by hand in the written exam. This is to ensure you're not just copying-and-pasting, but really understanding the concepts and coding approaches and being able to use them.

Course Syllabus

Fall Semester of AY 2025–2026

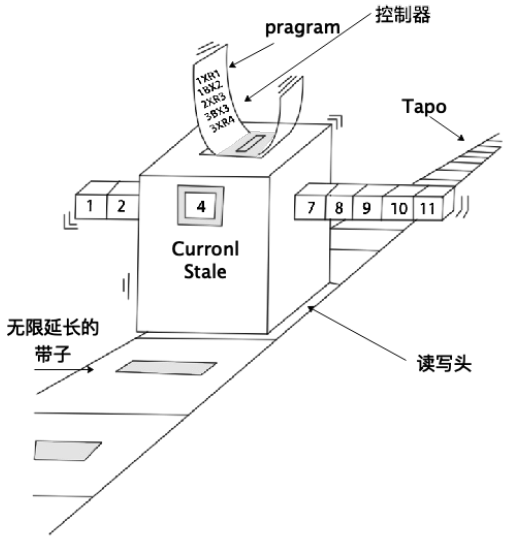
week \ month \ date \ day		Mon	Tue	Wed	Thur	Fri	Sat	Sun
Summer Session	2025 Aug.	11	12	13	14	15	16	17
		18	19	20	21	22	23	24
		25	26	27	28	29	30	31
1	Sept.	1	2	3	4	5	6	7
2		8	9	10	11	12	13	14
		15	16	17	18	19	20	21
		22	23	24	25	26	27	28
3		29	30					
				1	2	3	4	5
4	Oct.	6	7	8	9	10	11	12
5		13	14	15	16	17	18	19
6		20	21	22	23	24	25	26
7		27	28	29	30	31		
							1	2
8	Nov.	3	4	5	6	7	8	9
9		10	11	12	13	14	15	16
10		17	18	19	20	21	22	23
11		24	25	26	27	28	29	30
12	Dec.	1	2	3	4	5	6	7
13		8	9	10	11	12	13	14
14		15	16	17	18	19	20	21
15		22	23	24	25	26	27	28
16		29	30	31				
					1	2	3	4
17	2026 Jan.	5	6	7	8	9	10	11
18		12	13	14	15	16	17	18
Winter Break		19	20	21	22	23	24	25
		26	27	28	29	30	31	
								1
	Feb.	2	3	4	5	6	7	8
		9	10	11	12	13	14	15
		16	17	18	19	20	21	22
		23	24	25	26	27	28	

- **Lecture 1: Course Introduction**
- **Lecture 2: Basic Data Types in C Language**
- **Lecture 3: Data Input and Output**
- **Lecture 4: C Expressions and Macro Definitions**
- **Lecture 5: Selection Structures**
- **Lecture 6: Compilation Preprocessing**
- **Lecture 7: Loop Structure**
- **Lecture 8: Module Design**
- **Lecture 9: Arrays**
- **Lecture 10: Pointers (I)**
- **Lecture 11: Pointers (II)**
- **Lecture 12: Structures and Unions (I)**
- **Lecture 13: Structures and Unions (II)**
- **Lecture 14: Files**
- **Lecture 15: Bitwise Operation**
- **Course Summary and Review**

Course Introduction

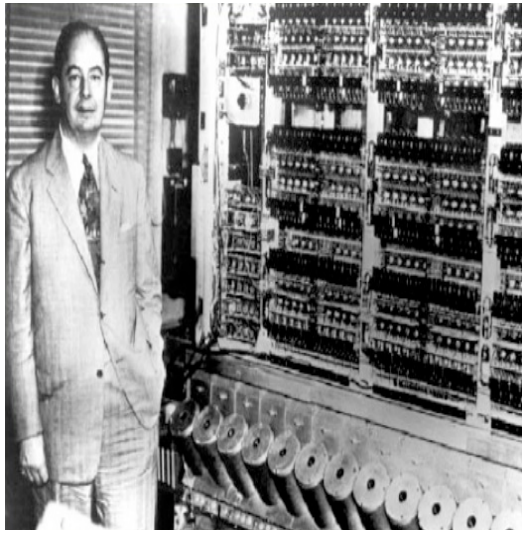
- 1.1 Fundamentals of computer and programming
- 1.2 Programming language
- 1.3 A simple C language program
- 1.4 C language program with Microsoft Visual Studio

Fundamentals of computer

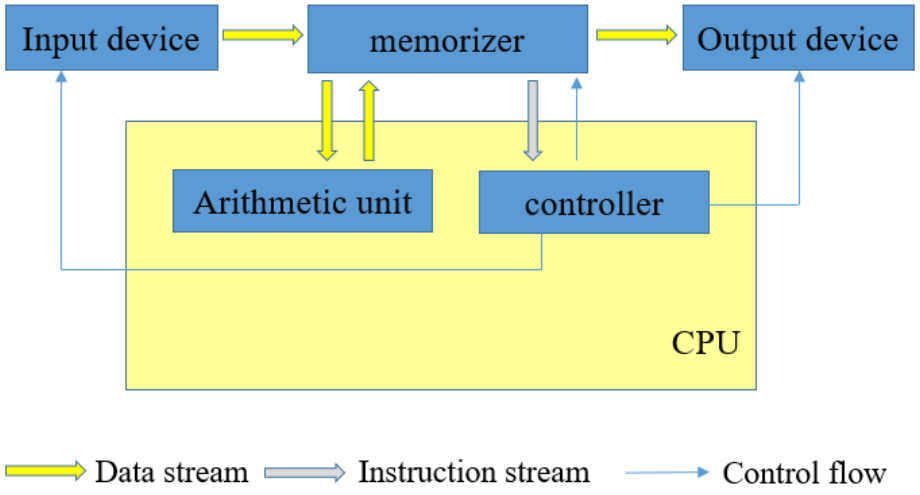


Alan Turing

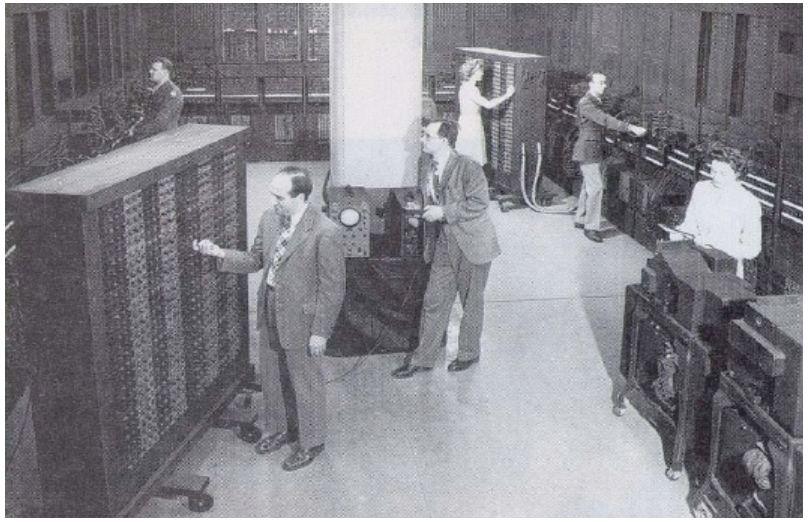
- The core idea of von Neumann architecture: digital computer with binary data format; the computer running the program in sequential mode.



Von Neumann



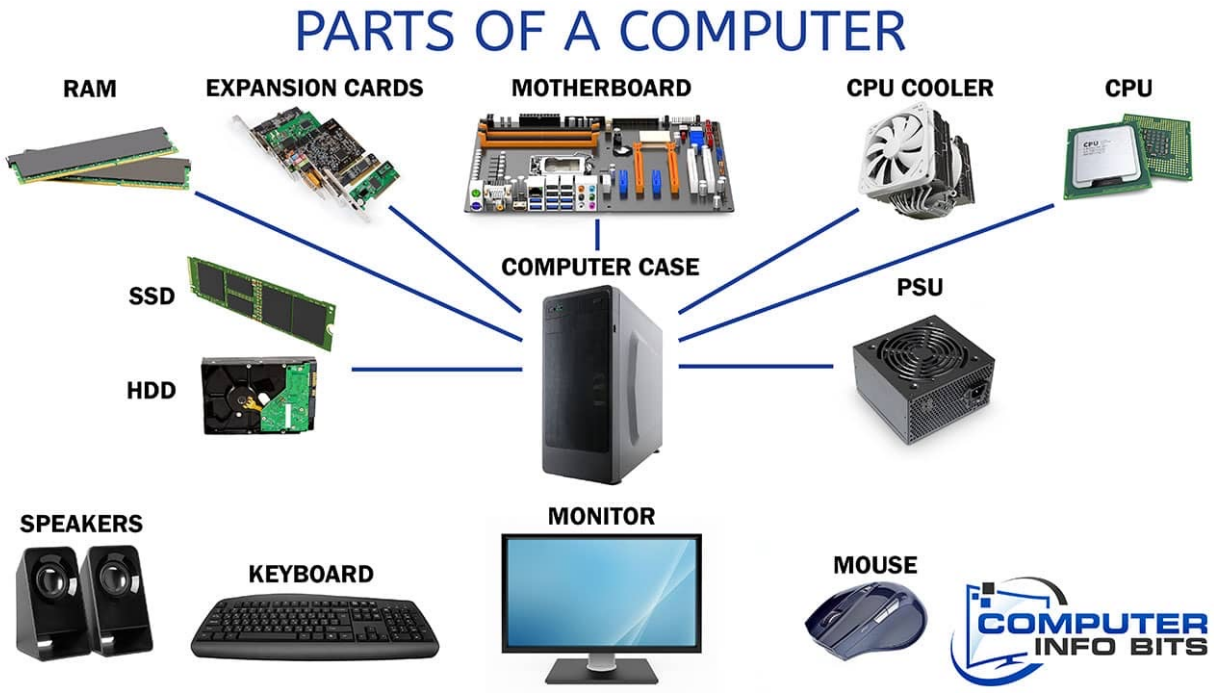
Von Neumann Architecture



The world's first digital electronic computer ENIAC

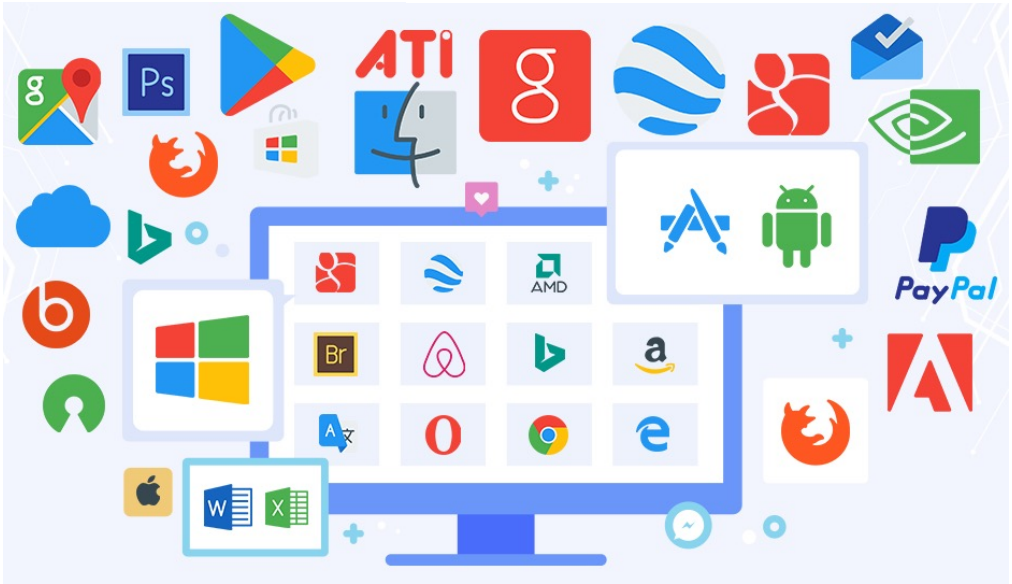
Fundamentals of computer

- **A computer is a system that can automatically and efficiently input, process, store and output data according to pre-stored programs. This system consists of hardware and software.**



Hardware

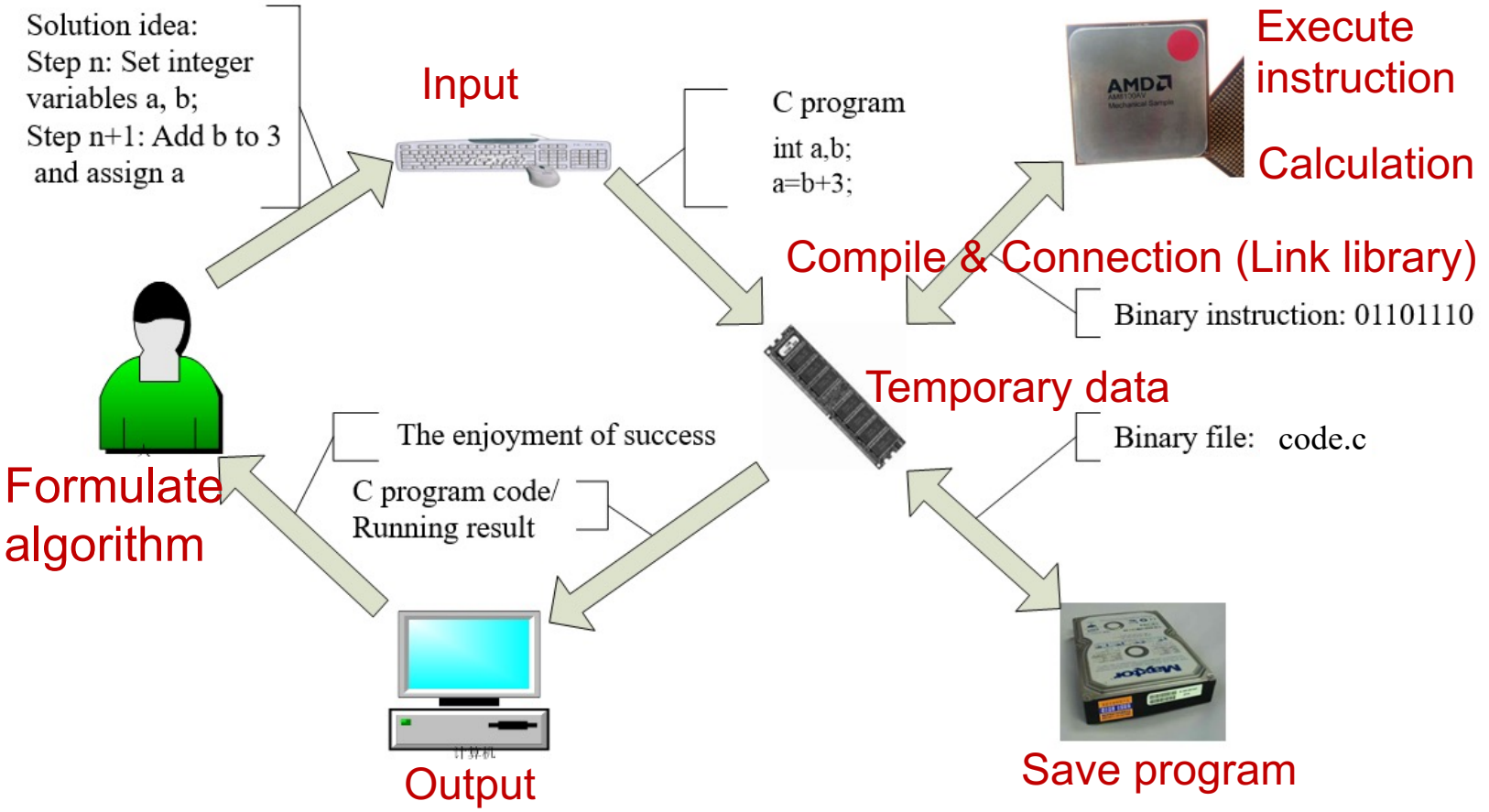
Visual Studio

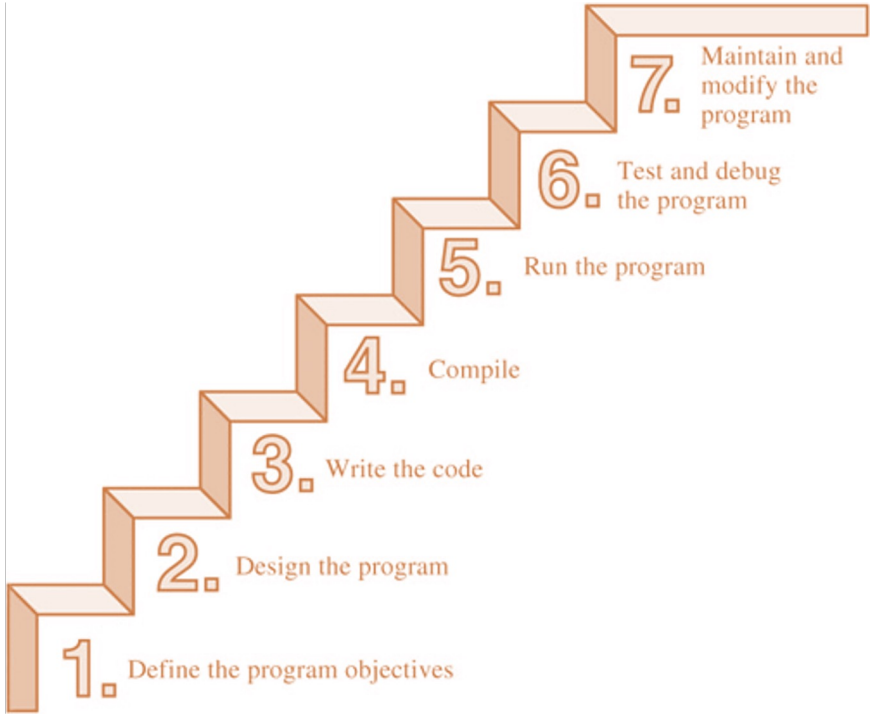


Software

Fundamentals of computer

- **A computer is a system that can automatically and efficiently input, process, store and output data according to pre-stored programs. This system consists of hardware and software.**





The core idea and basic ability of programming

Program concepts and core ideas

- A program is an ordered collection of instructions (Program Structure)
- Programs are stored sequentially in memory (Storage Type)
- CPU executes instructions in a programmatic structure (Algorithmic Process)
- Programs have input and output operations (Data I/O)

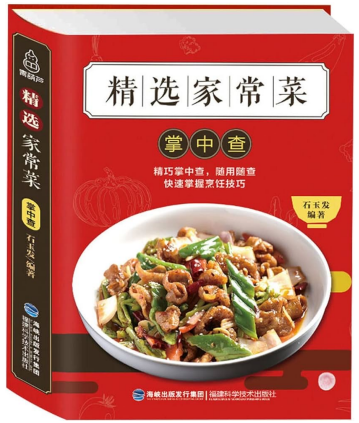
Programming process and basic capabilities

- Formulate algorithms: Problem analysis ability. e.g., Data and Algorithms course
- Editing code: Programming literacy. e.g., C, C++, Python, JAVA
- Compile link: Debugging capability. Compilation system, tool software
- Perform debugging: Testing capability. Datasets, software engineering

Overview of programming

- **Programming = algorithm + method + data structure + tool**

=



+

+

Source program (.cpp, .h) Executable program (.exe)	Problem solving ideas Common algorithms Program structure	Parameter types Return types Variable types	Project setup File & Library Compilation & Debug

Overview of programming

- **Programming = algorithm + method + data structure + tool**

  Include the following five basic steps:

  - **The analysis of the problem**
  - **Design of structural characteristics**
  - **Algorithm design**
  - **Description of the process**
  - **Debugging and running**

The problem analysis

- **Analysis of problem (The basics of programming):** According to the nature and type of problem to be solved, the most basic analysis content mainly has the following aspects:

  - **Nature of the problem:** Further identify what to do in the process of solving this problem? How to do?
  - **Input/output data:** (1) What is the type of data? Such as integer, real, double precision, character, and so on. (2) On which device is input or output made? (3) What format is used for data input or output?

- **Mathematical models or common methods:** For example, to solve the quadratic equation with one variable, we can use the root formula, Veda's theorem, iteration method, etc.

The design of the structural characteristics

- **Control structure:**
  - The function of a program depends not only on the selected operations, but also on the execution order among the operations, that is, the control structure of the program.
  - The control structure of the program actually gives the framework of the program and determines the execution order of the operations in the program.
  - In the process of programming, flow chart is usually used to represent the control structure of the program visually.

- **Data structure:** In general, when processing data, different forms of data organization, its processing efficiency is different.

The design of the structural characteristics

- **For example, the following table is a student information table, if you want to find students with 80 to 89 points, you need to determine whether the information in the table meets the requirements line by line.**

Student Number	Name	Gender	Age	Grade
80156	Xiaoming Zhang	male	20	86
80157	Xiaoqing Li	female	19	83
80158	Kai Zhao	male	19	70
80159	Qiming Li	male	21	91
80160	Hua Liu	female	18	78
80161	Xiaobo Zeng	female	19	90
80162	Jun Zhang	male	18	80
80163	Wei Wang	male	20	65
80164	Tao Hu	male	19	95
80165	Min Zhou	female	20	87
80166	Xuehui Yang	male	22	89
80167	Yonghua Lv	male	18	61
80168	Ling Mei	female	17	93
80169	Jian Liu	male	20	75

The design of the structural characteristics

- **However, if students with scores above 90, 80 to 89, 70 to 79, and 60 to 69 are registered in four separate tables, the search efficiency will be greatly improved.**

Student Number	Name	Gender	Age	Grade
80159	Qiming Li	male	21	91
80161	Xiaobo Zeng	female	19	90
80164	Tao Hu	male	19	95
80168	Ling Mei	female	17	93

Student Number	Name	Gender	Age	Grade
80156	Xiaoming Zhang	male	20	86
80157	Xiaoqing Li	female	19	83
80162	Jun Zhang	male	18	80
80165	Min Zhou	female	20	87
80166	Xuehui Yang	male	22	89

Student Number	Name	Gender	Age	Grade
80158	Kai Zhao	male	19	70
80160	Hua Liu	female	18	78
80169	Jian Liu	male	20	75

Student Number	Name	Gender	Age	Grade
80163	Wei Wang	male	20	65
80167	Yonghua Lv	male	18	61

When processing data, according to the different operations required, the data can be organized into a form that is convenient for operation in order to improve the efficiency of data processing.

Algorithm design

- In the process of problem analysis, the algorithm design involves either establishing a mathematical model or analyzing and comparing commonly used methods.

  An accurate and complete description of a solution. From a program point of view, an algorithm can also be said to be a finite set of instructions that determine the sequence of operations to solve a particular type of problem.

- Choosing a good algorithm is the key to programming. Two basic principles should be considered:

  (1) The cost of implementing the algorithm should be as small as possible, that is, the computation workload should be small.

  (2) The calculation results obtained by the algorithm should be reliable.

Description of the process

- The process of programming, in fact, is to determine the detailed steps to solve the problem, and these steps are often called program flow.

Description tool
Flow chart
Natural language
Algorithm description language
Programming

Example: Calculate and output z=y/x.

Flow chart

- In the practice of programming, people have summarized a set of graphics to describe the problem solving process, which makes the process more intuitive and acceptable.
- Flowchart: the tool that uses graphics to describe the processing flow.
- Currently, the commonly used flowcharts are the traditional flowcharts and structured flowcharts (also known as NS diagrams).

Traditional flow chart

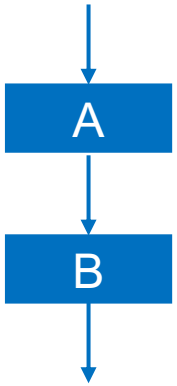
Structured program theorem (Bohm-Jacopini theorem, 1966): Any complex program can be composed of three basic structures: sequence, selection, and loop.

General and multi-case selection structure

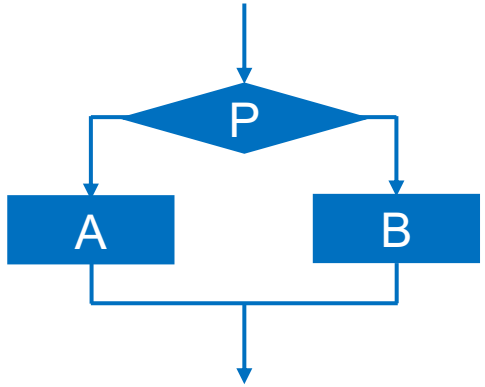
while and until loop structure

- **Sequence structure:** reflects the sequential order of execution among modules.
- **General selection structure:** the value of a condition determines which of the two modules is executed. **Multi-case selection structure:** according to the value of a control variable to decide which of the multiple modules to choose to execute.
- **While loop structure:** a specific module, i.e., the loop body, is executed repeatedly only when a certain condition is true. **Until loop structure:** a specific module is repeatedly executed until a certain condition is met, at which point the repeated execution of the module is exited.

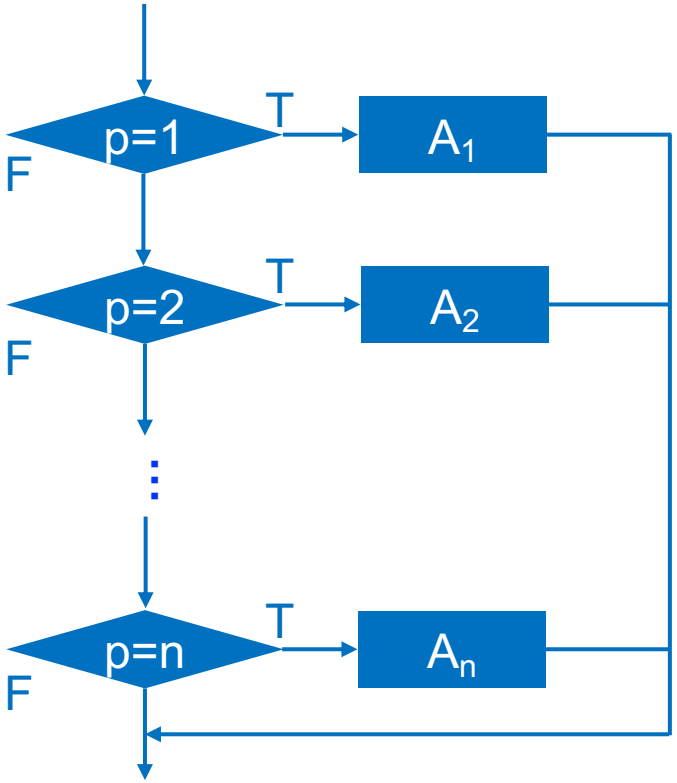
Traditional flow chart



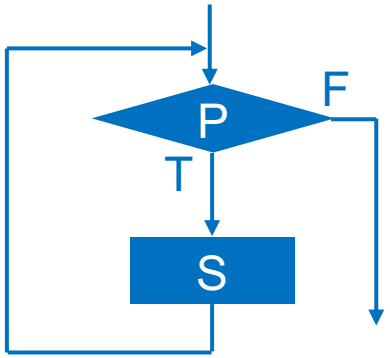
Sequential structure



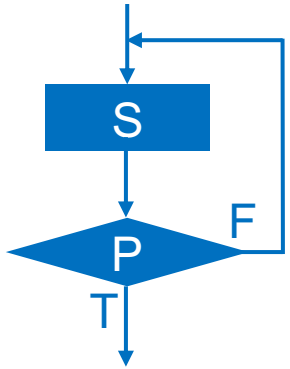
Selective structure



Multi-case selection structure (multiple selection structure)

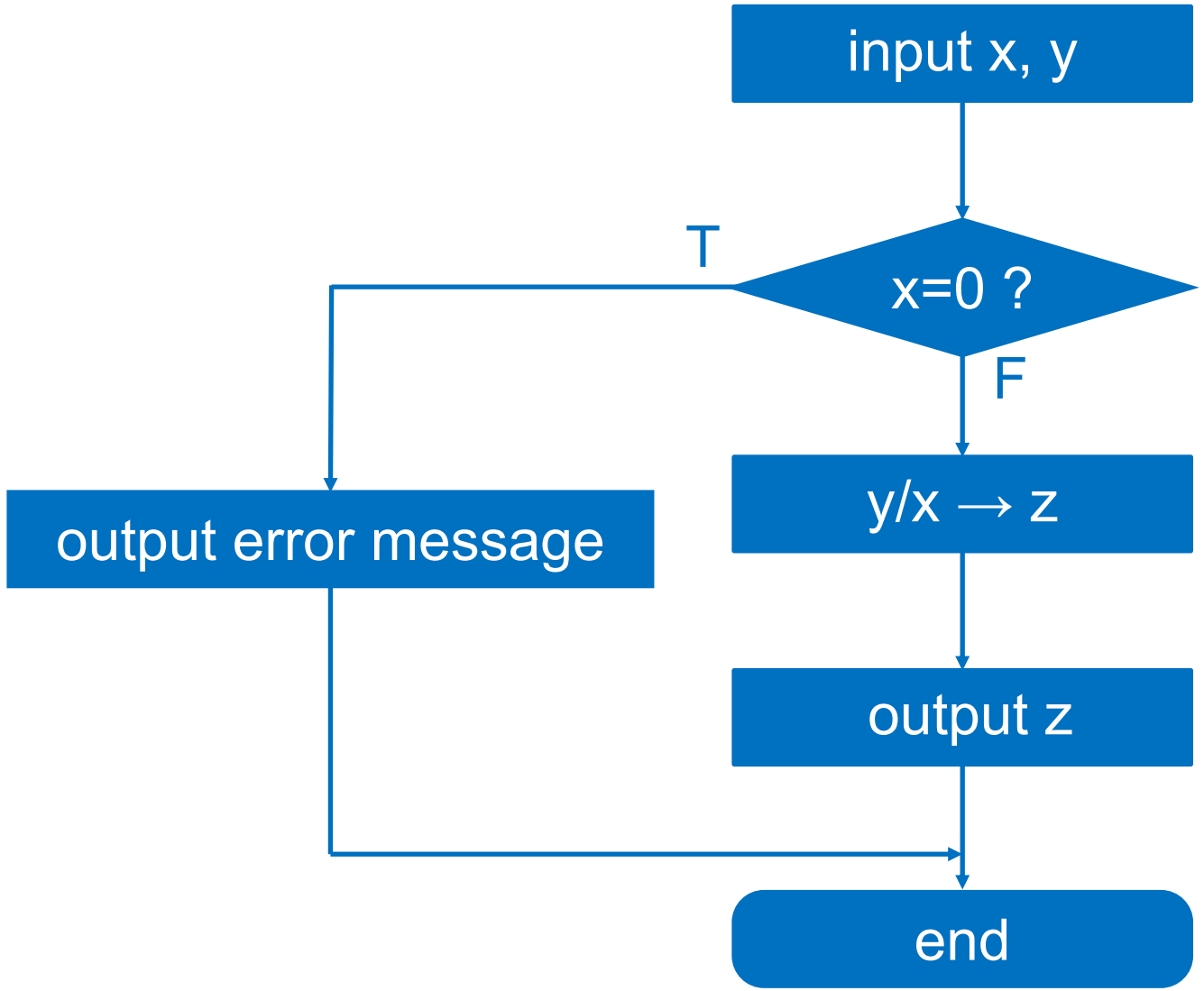


While loop



Until loop

Traditional flow chart



Traditional flow chart

Traditional flow charts have the following major disadvantages:

- **The traditional flowchart is not essentially a good tool for step-by-step refinement. It makes programmers consider the control flow of the program too early, without thinking about the overall structure of the program.**
- **Traditional flowcharts are not easy to represent hierarchical structures.**
- **Traditional flowcharts are not easy to represent important information, such as data structures and module call relationships.**
- **Arrows are used to represent control flow in traditional flowcharts. Therefore, programmers are not bound by any constraints and can freely transfer control, completely ignoring the idea of structured programming.**

Structured flow chart (NS Graph)

- **Structured programming requires limiting the structure of programs to three basic structures: sequence, selection, and loop, in order to improve the readability of the program.**

- **Two characteristics: (1) It simplifies interfaces and enhances comprehensibility by having a single entrance and exit per control structure unit. (2) It also minimizes the gap between the program's static structure and dynamic execution, facilitating accurate understanding of its function.**

- **The NS diagram (box diagram), invented by I. Nassi and B. Schneiderman, is a graphical tool that adheres to structured principles. It eliminates complex flow lines, representing algorithms and their basic structures within boxes.**

Structured flow chart (NS Graph)

Sequential structure

S_1
S_2
S_3

Selective structure: Two branch structure

Conditions	
Satisfy	Dissatisfy
S_1	S_2

Selective structure: Multiple branching structure

Conditions			
Case 1	Case 2	…	Case n
S_1	S_2	…	S_n

Loop structure: while type loop

While conditions
S

Loop structure: until type loop

S
Until conditions

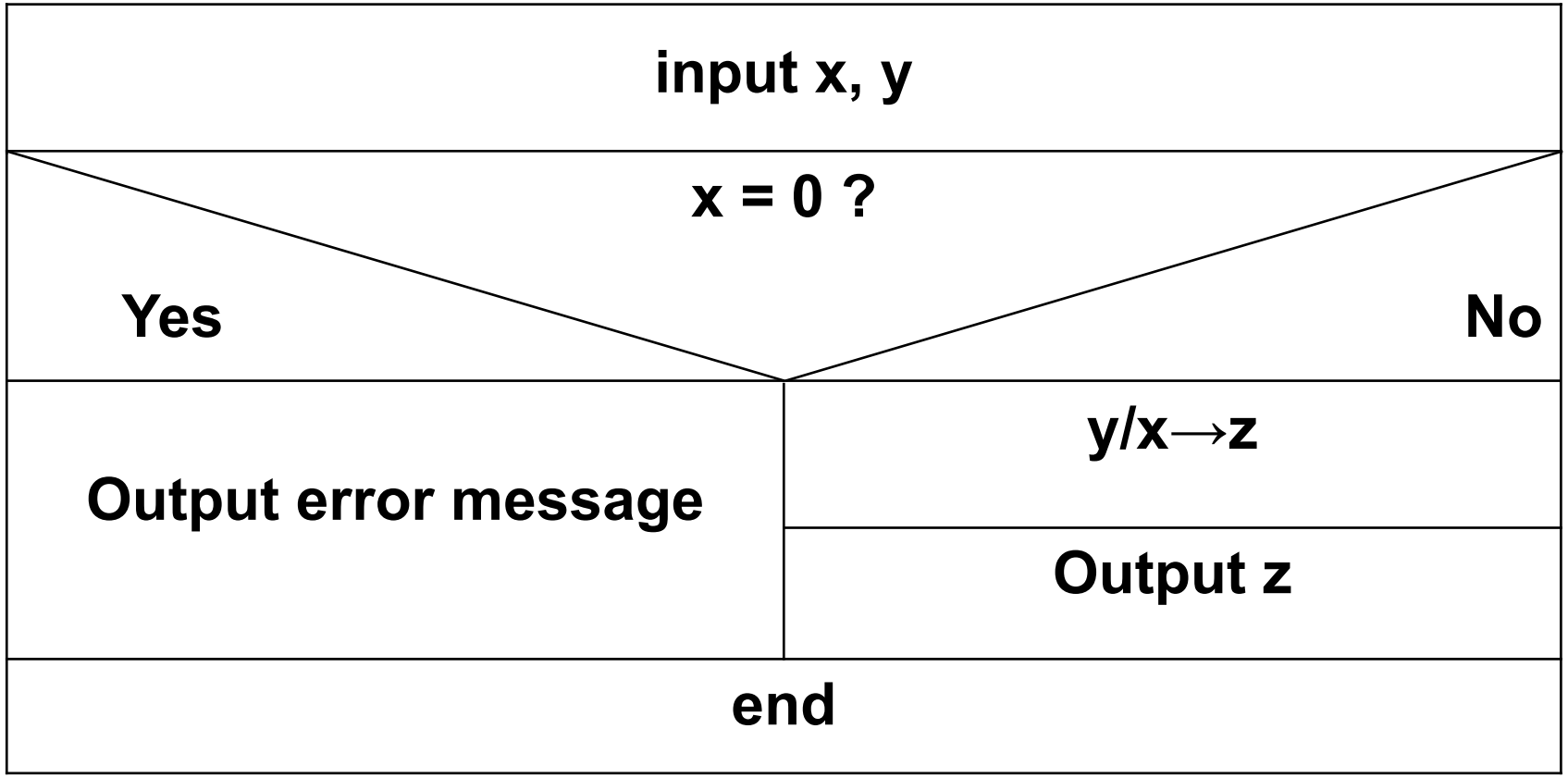
- **In the loop body S, there should be a component to change the condition, otherwise it will cause an endless loop.**

Structured flow chart (NS Graph)

- **Each module, such as S, S1, S2, S3, etc., can be nested as one of these three basic structures.**
- **The NS diagram has the following basic characteristics:**
  - **The functional domain is relatively clear and can be directly reflected in the block diagram.**
  - **It is not possible to arbitrarily transfer control, which conforms to the principle of structuration.**
  - **It is easy to determine the scope of local and global data.**
  - **It is easy to represent nested relationships and can also represent the hierarchical structure of modules.**

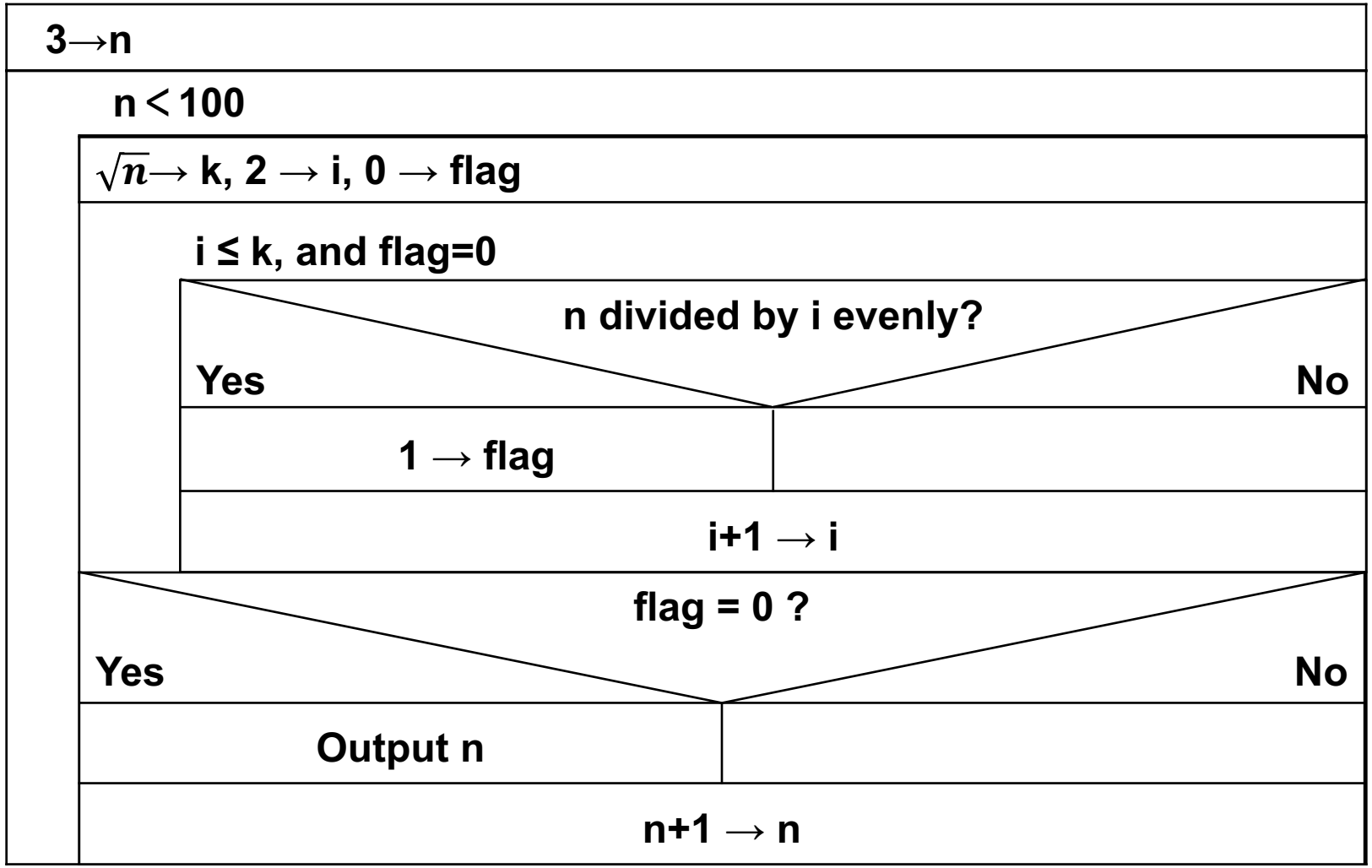
Structured flow chart (NS Graph)

- **For example, calculate and output z=y/x. NS diagram is used to describe its program flow as follows:**



Structured flow chart (NS Graph)

- Another example: find all prime numbers between 3 and 100 with NS graph



Structured flow chart (NS Graph)

- **Example 1: calculate and output the sum of series**

$$sum = 1 - \frac{1}{3} + \frac{1}{5} - \cdots + \frac{(-1)^K}{2K+1} + \cdots$$

until the absolute value of a term is less than 10^{-4}.

- **Let f be a switching variable that only takes the values {1, -1} to change the sign of each term. This is a series with alternating signs.**
- **NS diagram:**

sum = 1.0, k = 0, f = 1	
	k = k + 1, f = -f d = 1.0/(2×k+1) sum = sum + f×d
until $d < 10^{-4}$	
output sum	

Structured flow chart (NS Graph)

- **Example 2: calculate and output the sum of series (Taylor series expansion of the exponential function e^x at the point $x = 0$)**

$$S_n = 1 + 0.5x + \frac{0.5(0.5-1)}{2!}x^2 + \frac{0.5(0.5-1)(0.5-2)}{3!}x^3 + \cdots + \frac{0.5(0.5-1)\ldots(0.5-n+1)}{n!}x^n$$

until the $|S_n - Sn_{-1}| < 0.000001$. The value of x is read from the keyboard.

- **Let t_{n-1} be the value of the (n-1)-th term, the value of the n-th term t_n can be calculated using the recursive relationship: $t_n = t_{n-1} \times (0.5\text{-}n+1) \times x/n$, where the initial value of t_n is 1.**

- **NS diagram:**

input x	
s = 1.0, n = 1, t = 1.0	
	t = t× (0.5-n+1)×x/n s = s + t n = n + 1
until |t|< 0.000001	
output s	

Natural language

- **Calculate and output z=y/x. The process can be described in natural language as follows:**

> **Step 1: Input x and y.**
>
> **Step 2: Check if x is 0;**
>
> **If x = 0, output an error message;**
>
> **Otherwise, calculate y / x → z, and output z.**

- **Describing algorithm flows in natural language does not follow a unified pattern, and it can be too arbitrary and loosely expressed. Therefore, it is not recommended to adopt this method.**

Algorithm description language (Pseudocode)

- **Calculate and output z=y/x.** The process can be described in PASCAL-like algorithm description language as follows:

```
INPUT x, y
IF (x=0) THEN
   OUTPUT "ERROR”
ELSE
{
   z=y/x
   OUTPUT z
}
```

- **Augmenting programming language with natural language descriptions.** Describing the algorithm process using algorithm description language is relatively strict in expression and thus can be adopted.

Programming

- **A program written in C to compute z=y/x:**

```
#include <stdio.h>
int main() {
    float x, y, z;
    printf("input x, y:");          /* Input hints */
    scanf_s("%f%f", &x, &y);        /* Read the values of x and y */
    if (x == 0)
        printf("error! x=0\n");     /* If x=0, an error message is displayed */
    else {                          /* Otherwise calculate and output the result */
        z = y / x;
        printf("z= %f\n", z);
    }
    return 0;
}
```

Debugging and running

- **Testing:** It refers to finding as many errors in the program as possible through some typical examples. Therefore, the purpose of testing is to discover errors in the program, not to prove the correctness of the program.

- **Debugging:** It refers to identifying the specific location of errors in the program and correcting them. Therefore, debugging is to find and fix errors.

- **Testing and debugging often alternate.** Errors in the program are found through testing, and the location of the errors is further identified and corrected through debugging. This process may need to be repeated multiple times.

Course Introduction

- 1.1 Fundamentals of computer and programming
- 1.2 Programming language
- 1.3 A simple C language program
- 1.4 C language program with Microsoft Visual Studio

Programming language

- **Computers are usually commanded and manipulated by humans.** In order to solve practical problems with computers, programs are generally required.

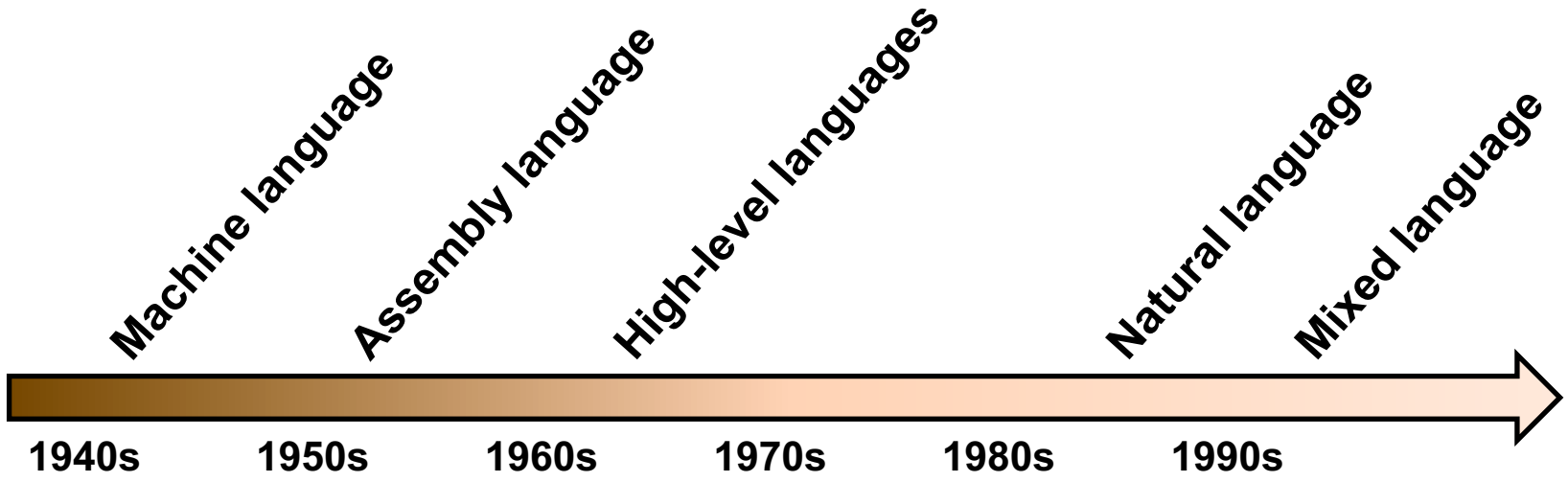
- **Program:** it refers to a sequence of actions created using a programming language as a tool. It represents people's approach to solving problems and directs the computer to perform a series of operations, thereby achieving predetermined functions.

- **Programming language:** the language used by users to write programs. It serves as a tool for exchanging information between humans and computers. It is a crucial component of the computer software system, and various corresponding language processing programs belong to system software.

- **Different types of computer languages:** Computer languages can be divided into machine language, assembly language, and high-level language.

The development of computer languages

- As a direct means of operating computers, computer languages have gone through several eras of development

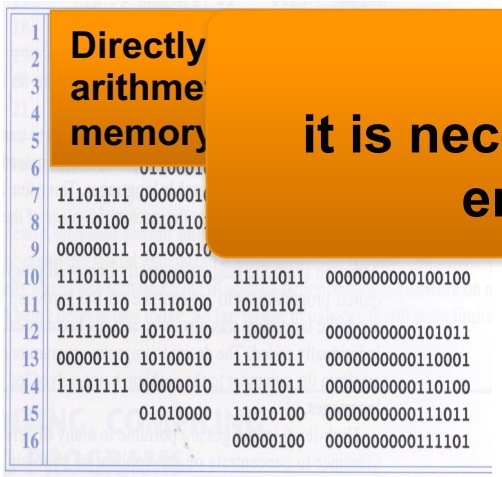


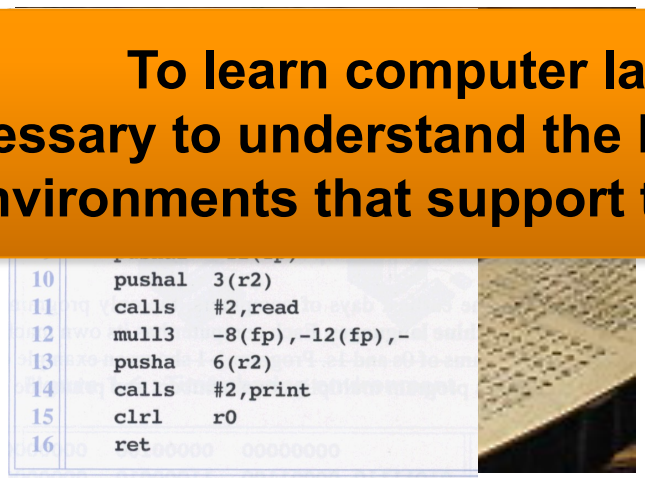
Directly
arithme
memory

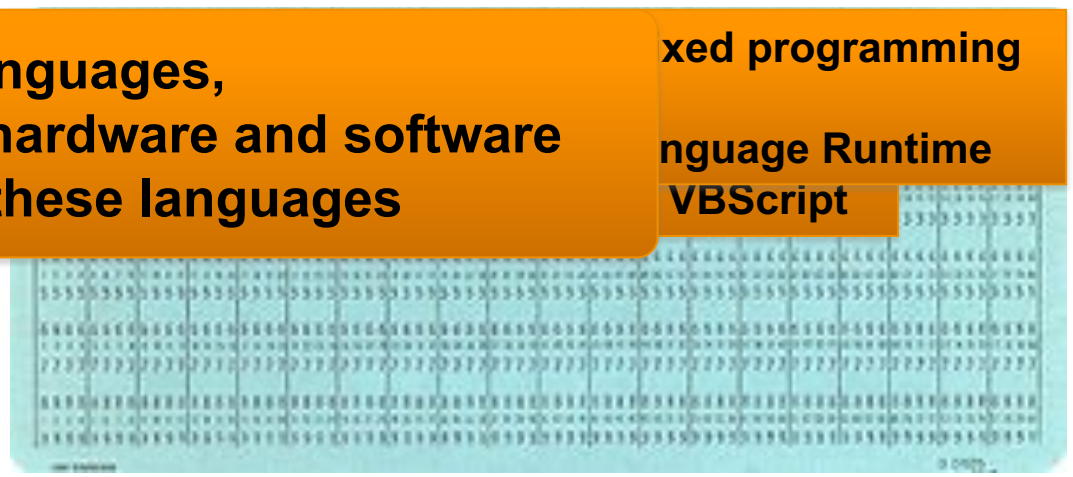
xed programming

nguage Runtime
VBScript

To learn computer languages,
it is necessary to understand the hardware and software environments that support these languages







Machine language

- A set of machine instructions is called a machine language program
- Machine language is the lowest level of computer language
- Computer can directly recognize programs written in machine language
- Of course, computer hardware only understands machine language
- Each machine instruction is a binary instruction code
- Instruction code: (1) The opcode tells the computer what kind of operation to perform; (2) The address code indicates the object to be operated on

- The instruction system varies among different computer hardware
- Written specifically for certain computer hardware
- Have high execution efficiency to maximize the computer's performance
- Difficult to program, error-prone, and lacks intuitiveness and readability
- Not easy to port to other platforms

Machine language

- **Example: Machine language program written with Intel 8086 instruction set to calculate 5+3:**

```
01010101
10001011  11101100
01001100
01001100
01010110
01010111
10111111  00000101  00000000
10111110  00000011  00000000
10001011  11000111
00000011  11000110
10001001  01000110  11111110
01011111
01011110
10001011  11100101
01011110
11000011
```

Assembly language

- To facilitate memory, (1) use English abbreviations (mnemonic symbols) to replace opcodes, (2) use address symbols to replace address code
- Assembly or symbolic instructions: instructions composed of mnemonic symbols and address symbols
- Assembly or symbolic language source programs: programs written with assembly or symbolic instructions

- It's worth noting that the computer can't directly understand assembly language programs. They must be translated into machine language via a special program called an "assembler" before execution. This translation is termed "assembly".

Assembly language

- **Example: write an assembly language program and its corresponding machine language program for calculating 5+3 using the Intel 8086 instruction set**

PUSH BP	01010101
MOVE BP, SP	10001011 11101100
DEC SP	01001100
DEC SP	01001100
PUSH SI	01010110
PUSH DI	01010111
MOVE DI, 0005	10111111 00000101 00000000
MOVE SI, 0003	10111110 00000011 00000000
ADD AX, SI	10001011 11000111
MOVE AX, DI	00000011 11000110
ADD AX, SI	10001001 01000110 11111110
MOVE [BP-02], AX	01011111
POP DI	01011110
MOVE SP, BP	10001011 11100101
POP BP	01011110
RET	11000011

High-level language

- **Low-level languages:** Machine language and assembly language are both machine-oriented languages, referred to as low-level languages.
- **High-level languages:** Since the mid-1950s, problem-oriented programming languages have gradually developed, called high-level languages.

- The main goals of program design are readability, maintainability, and portability of the program.

- Any high-level language program (source code) must be compiled into machine code (target program) for computer execution, or interpreted on-the-fly (e.g., via Basic, Python). Programming languages are evolving to be more like human language.

High-level language

- Using a high-level language simplifies calculating "5 + 3"

- Programs written in BASIC language:

```
10    I=5
20    J=3
30    K=I+J
```

- Programs written in Pascal language:

```
Program Addit
    var
      i, j, k : integer;
    begin
      i:=5;
      j:=3;
      k:=i+j;
    end
```

- Programs written in C language:

```
#include <stdio.h>
main()
{
        int i, j, k;
        i = 5;
        j = 3;
        k = i + j;
}
```

Course Introduction

- 1.1 Fundamentals of computer and programming
- 1.2 Programming language
- 1.3 A simple C language program
- 1.4 C language program with Microsoft Visual Studio

Simple C language program

- **Example 1-1: Write a C program that displays strings “Hello, World!”.**
- **Its C program is as follows:**

```
#include <stdio.h>
main()
{
        printf("Hello, World!\n");
}
```

Newline

- **This is a simple yet complete C language program. If this program is entered into the computer using editing software, compiled and linked correctly through the compiler, running the executable file of this program will display the following text at the current cursor position on the screen:**

```
Hello, World!
```

- **Note: Use <stdio.h> instead of "stdio.h" after #include**

Simple C language program

- **Example 1-2: The function of the following C language program is to input two double precision real numbers from the keyboard, and then calculate and display the square root of the sum of the squares of these two real numbers.**

```
#include <stdio.h>
#include <math.h>
main()
{
    double x, y, s; /* Defines three real variables*/
    printf("input x and y : "); /* Gives input prompt*/
    scanf_s("%lf,%lf", &x, &y); /* Enter x and y values  */
    s = sqrt(x * x + y * y); /* Sqrt is a library function in math*/
    printf("s = %f\n", s); /* Output result*/
}
```

- **Program result:**

  input x and y: 3,4

  s=5.000000

Simple C language program

- A C program can have multiple functions but requires a unique main function, from which execution begins. Functions serve as modular units in C.
- In a C function module, the part enclosed by left and right curly braces {} is the function body, where a series of statements implement the intended functionality of the function.
- In C programming, statements must end with a semicolon “;”, but formatting is flexible, allowing multiple statements on one line or one statement across multiple lines. Readability should be a priority.
- #include is a preprocessing directive that inserts file content, enclosed in angle brackets or quotes, into the code.
- Comments can be added anywhere in a C program using /* ... */ to improve the readability of the program.

C program composition

Macro definition, defining constant N, defining macro function
File inclusion, including system files, including user files
External variables (global variables, file-scope local variables)
Function forward declaration
Main function with command line arguments
{
    Function forward declaration
    Local variables
    Function calls
    Dynamic memory allocation
    Calling macro-defined functions
    File operations
    Input and output, formatting issues
}

C language program on the computer steps

- Use editing software to input C source program and create a C source program file. The extension of the C source program file is .c.
- Compile the C source code to generate an .OBJ file, then link it to create an .EXE file. If errors arise, edit the source code and recompile/relink until no errors remain.
- Running the executable yields results. If errors occur, recompile object files, edit the source of called functions, then recompile, relink, and rerun until error-free.

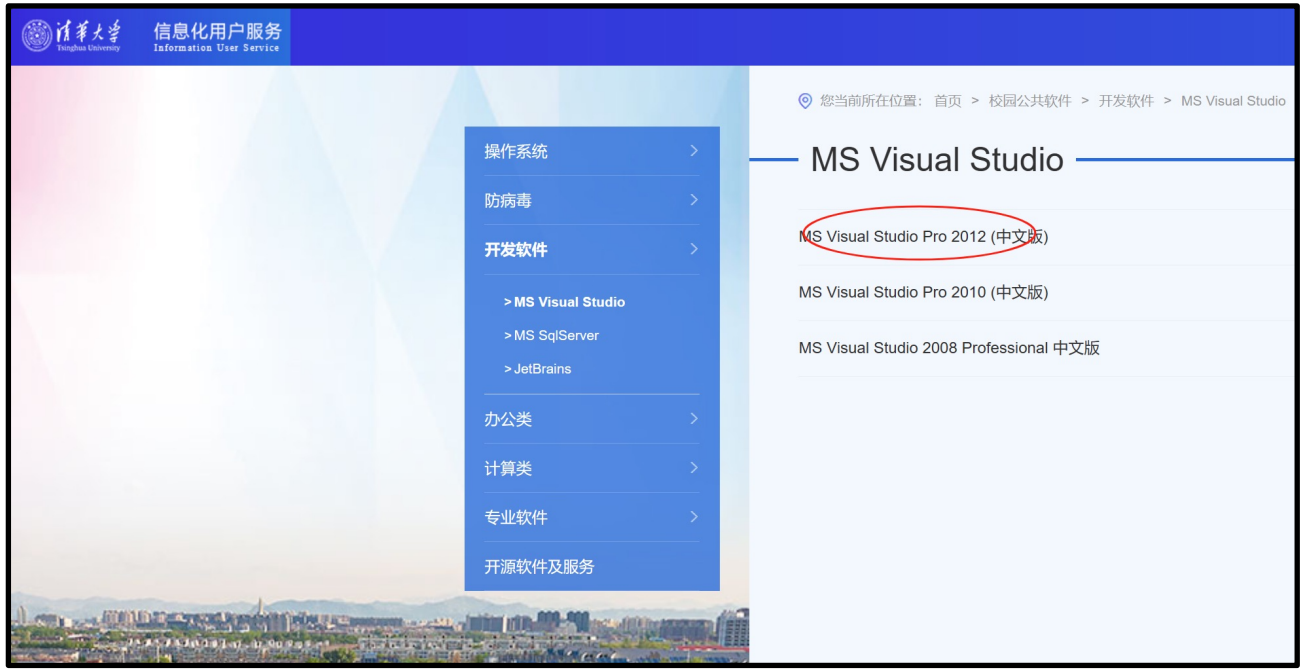
- Link compiled object files and function modules to create an executable file.
- Analyze source program statements, translate to machine instructions, and detect errors.

Course Introduction

- 1.1 Fundamentals of computer and programming
- 1.2 Programming language
- 1.3 A simple C language program
- 1.4 C language program with Microsoft Visual Studio

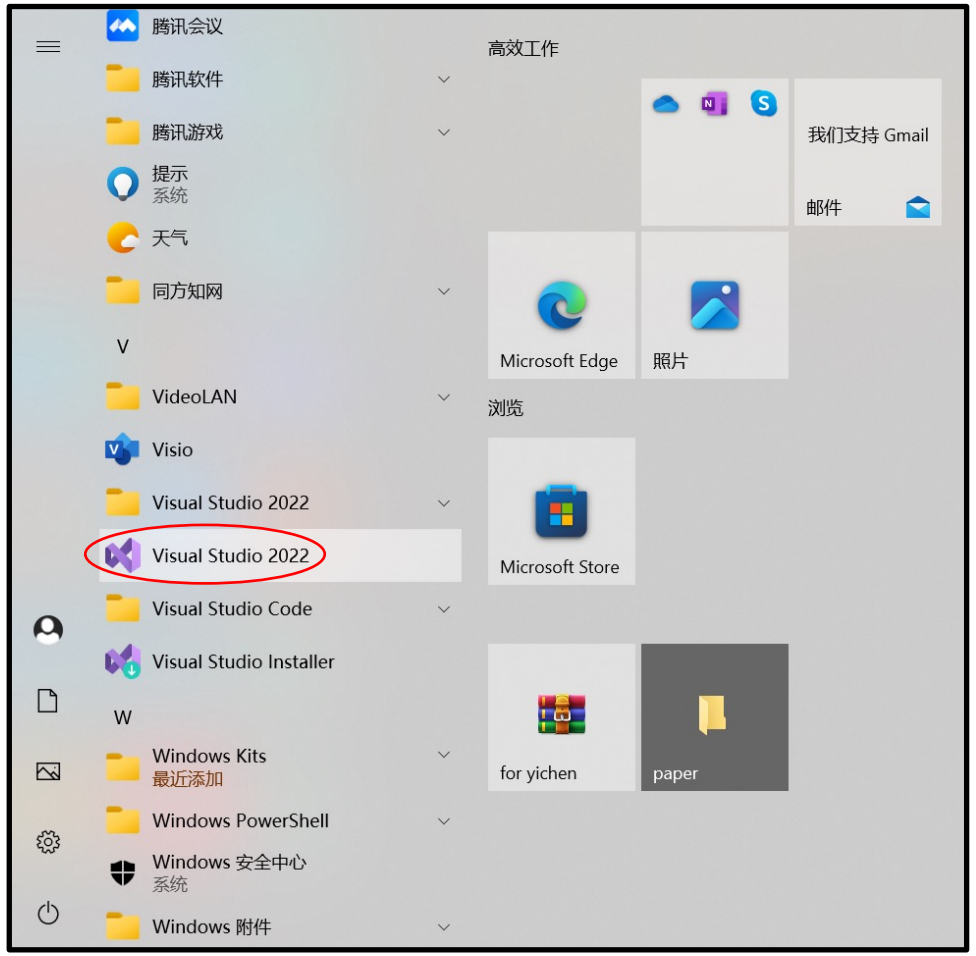
Visual Studio Guide

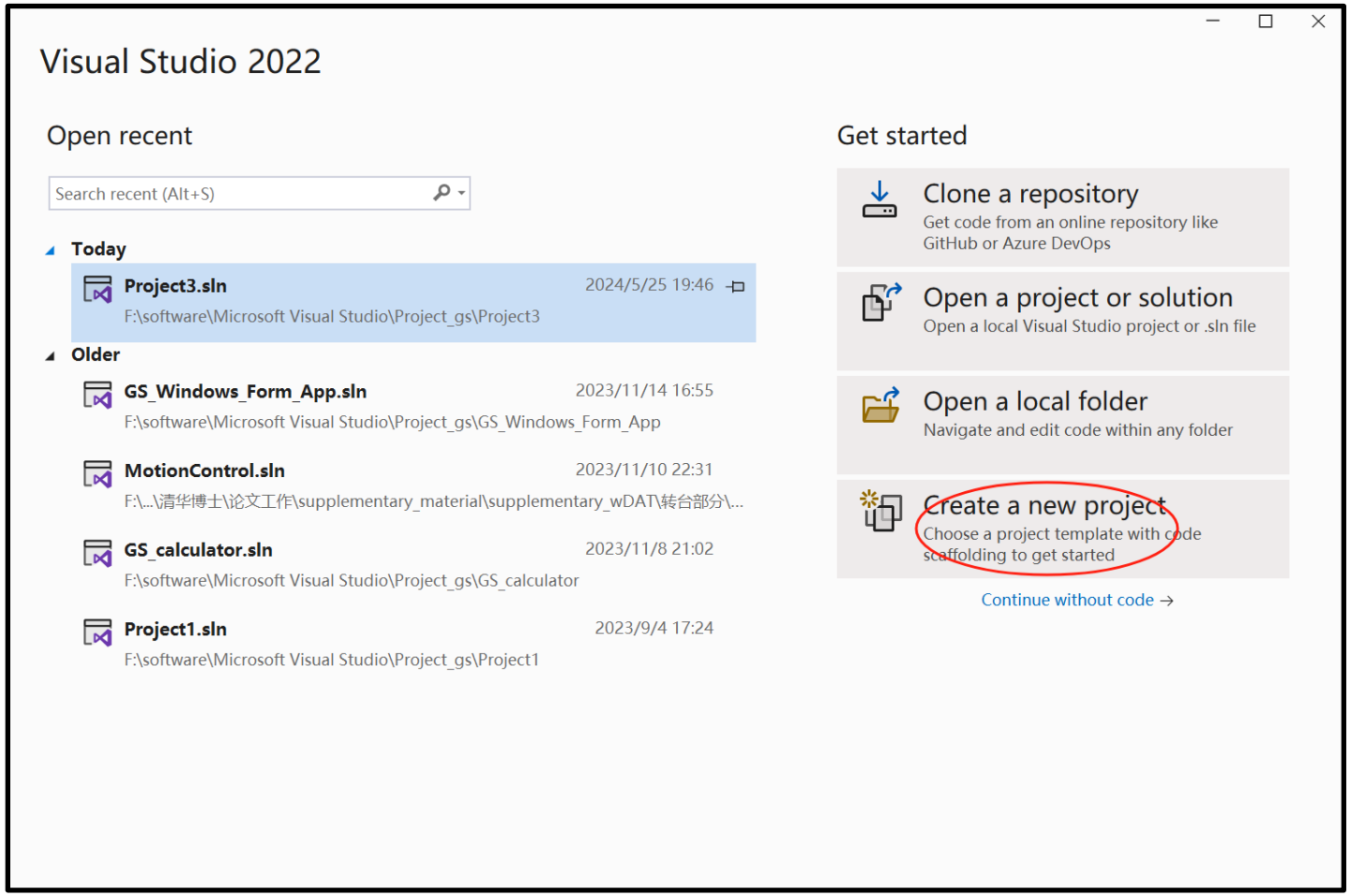
- **Download VS from the Microsoft official website or the software resources in the Tsinghua information user service (https://its.tsinghua.edu.cn/).**
- **Install the software**



Visual Studio Guide

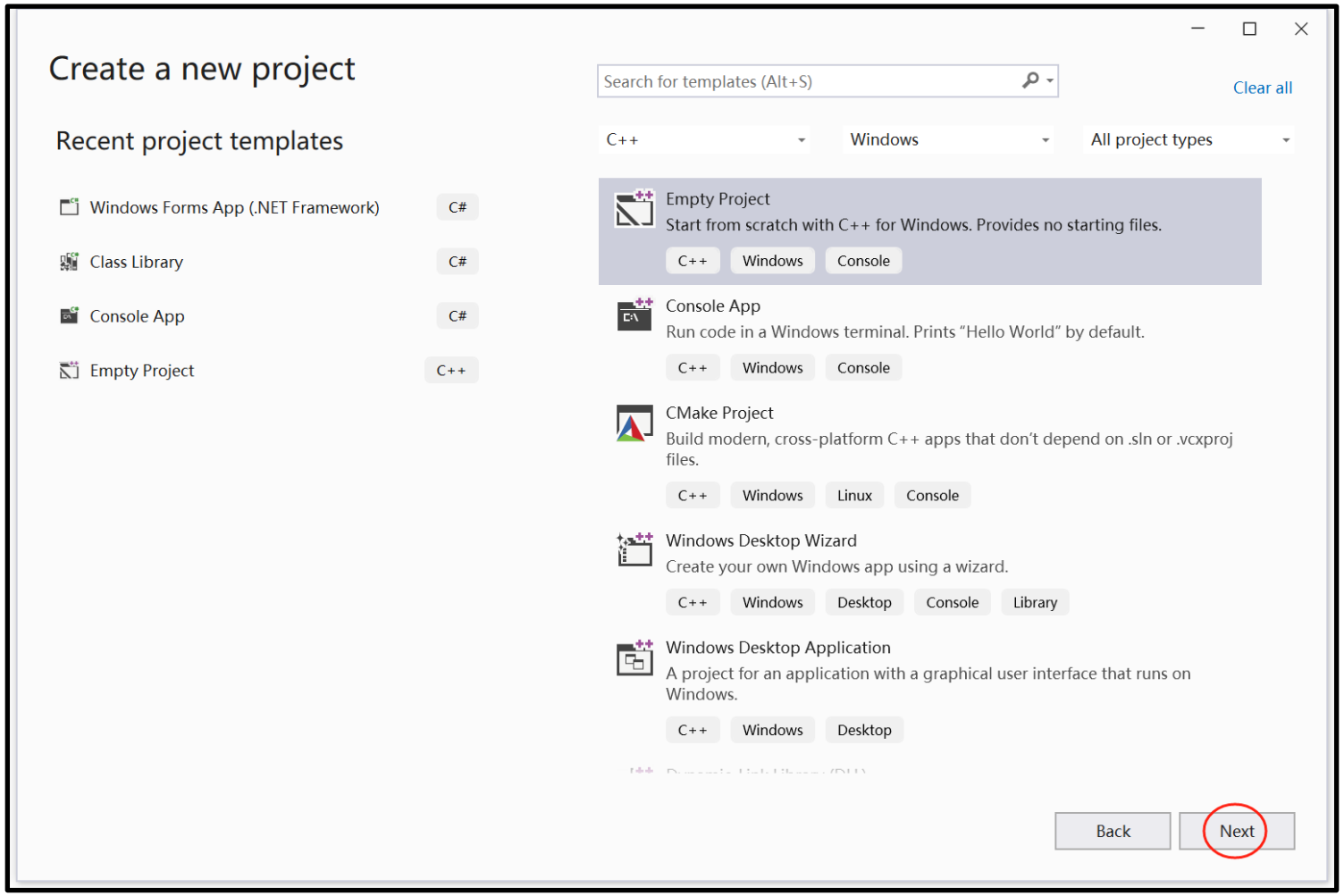
- To start VS2022, select Start → programs → Microsoft Visual Studio 2022





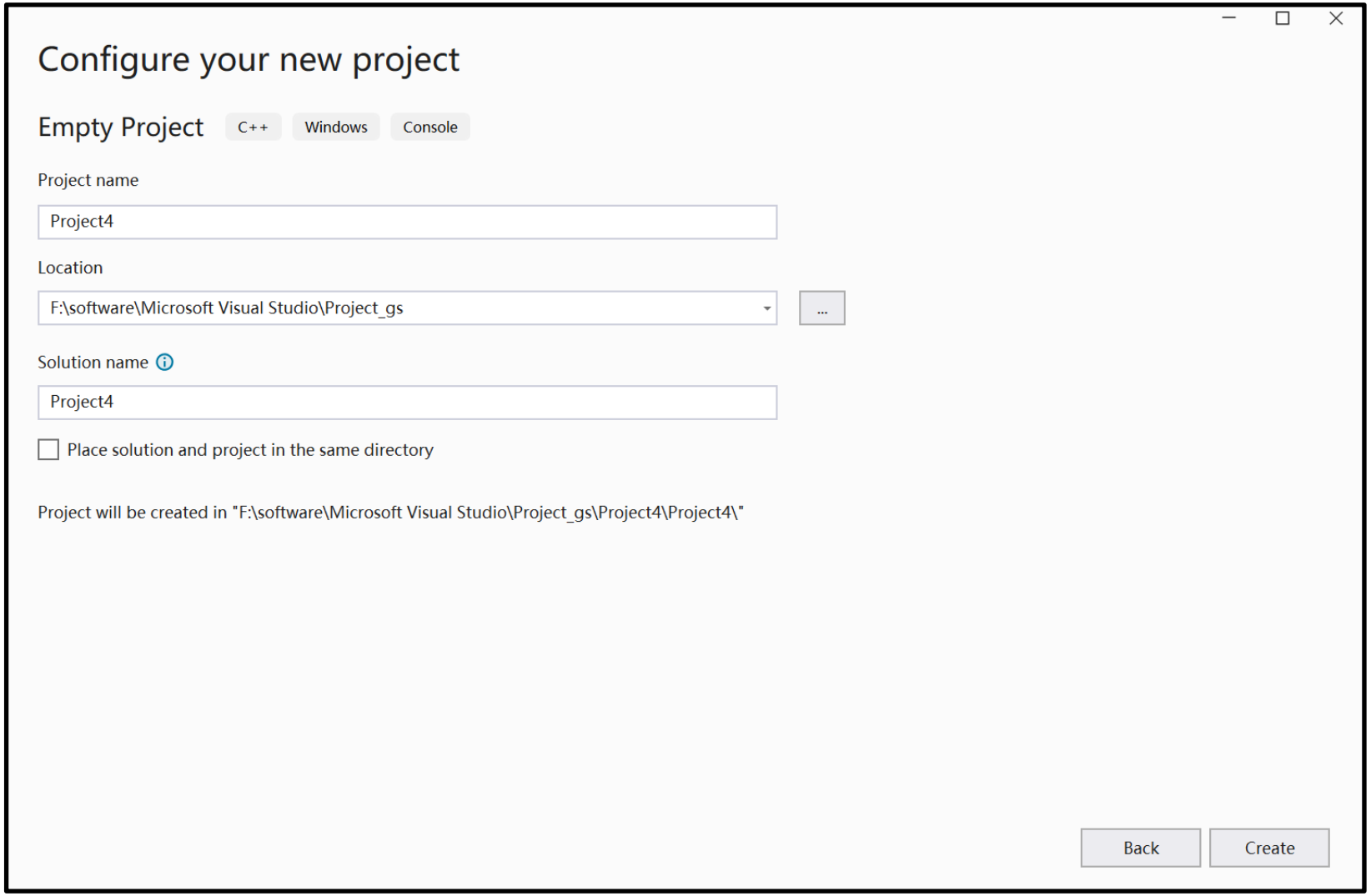
Visual Studio Guide

- **Create a new project → C++ → Empty Project → Next**



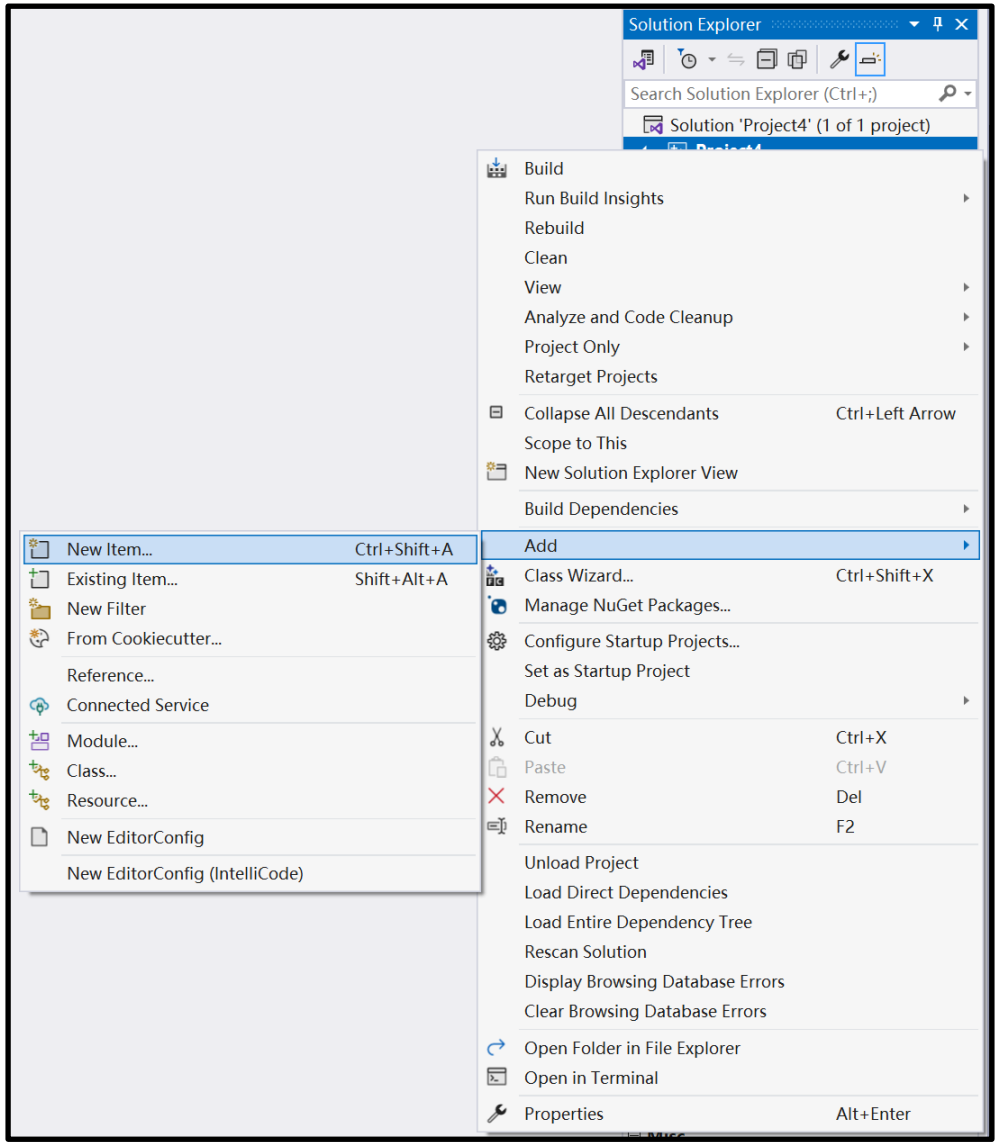
Visual Studio Guide

- **Create a project called Project4 and select the location of Project → Create**



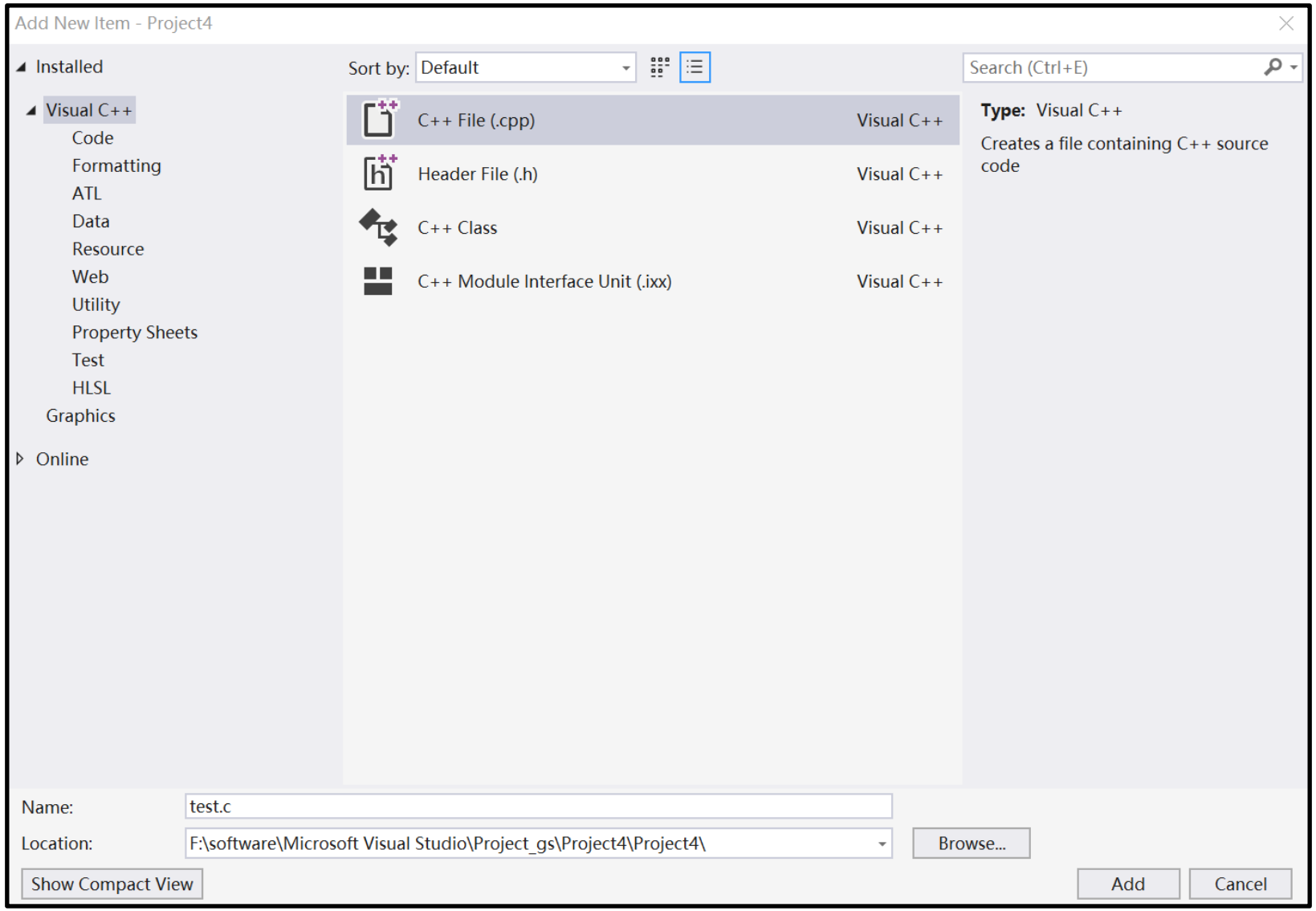
Visual Studio Guide

- **Right click on Project4 → Add → New Item**



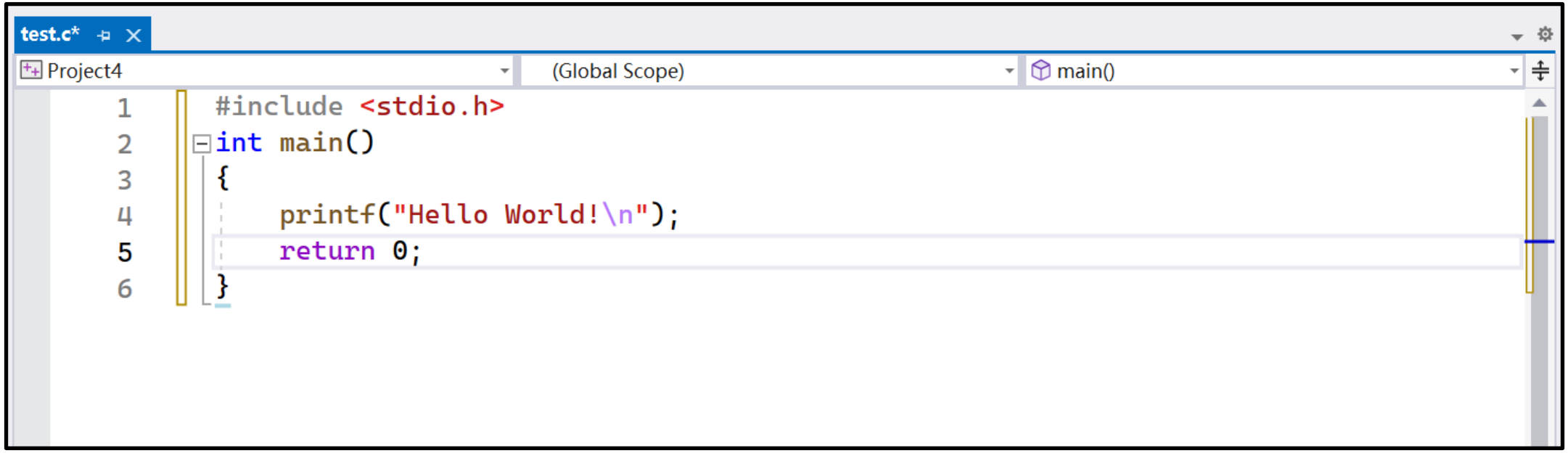
Visual Studio Guide

- **Select C++ File → Set Name: test.c → Add**



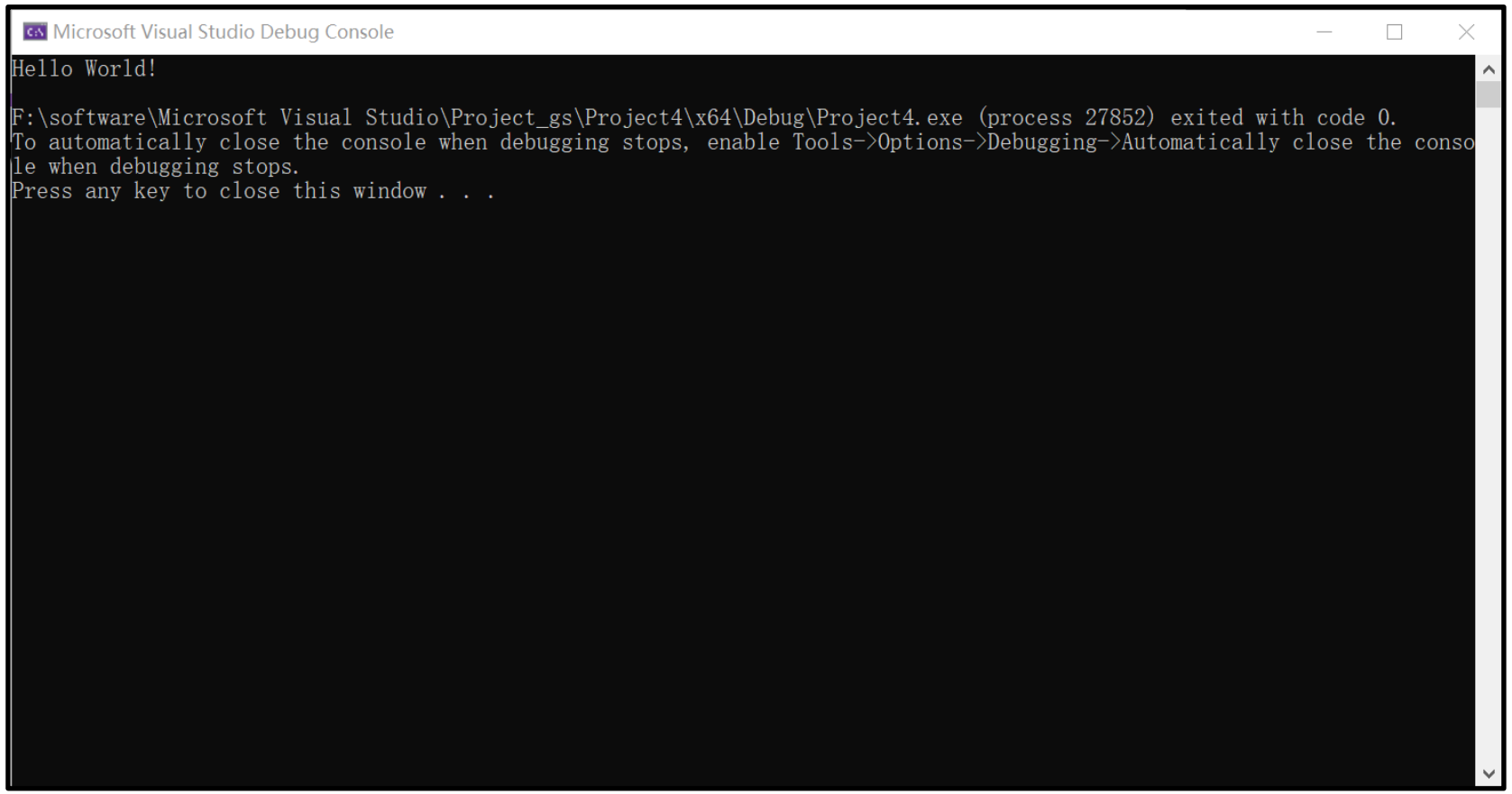
Visual Studio Guide

- **Enter the program code.**



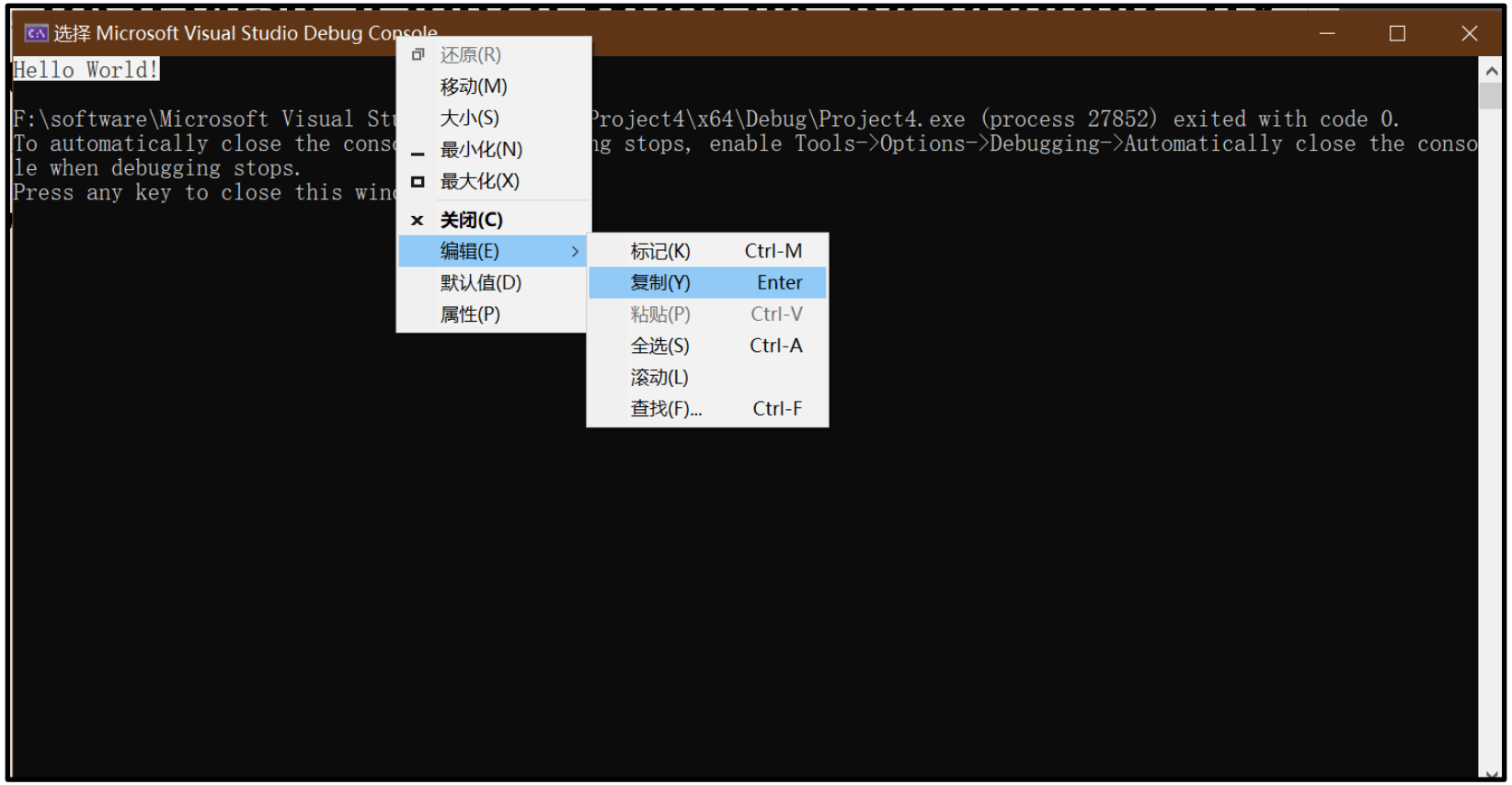
Visual Studio Guide

- **Execute the program: ("Local Windows Debugger")**
- **Running result:**



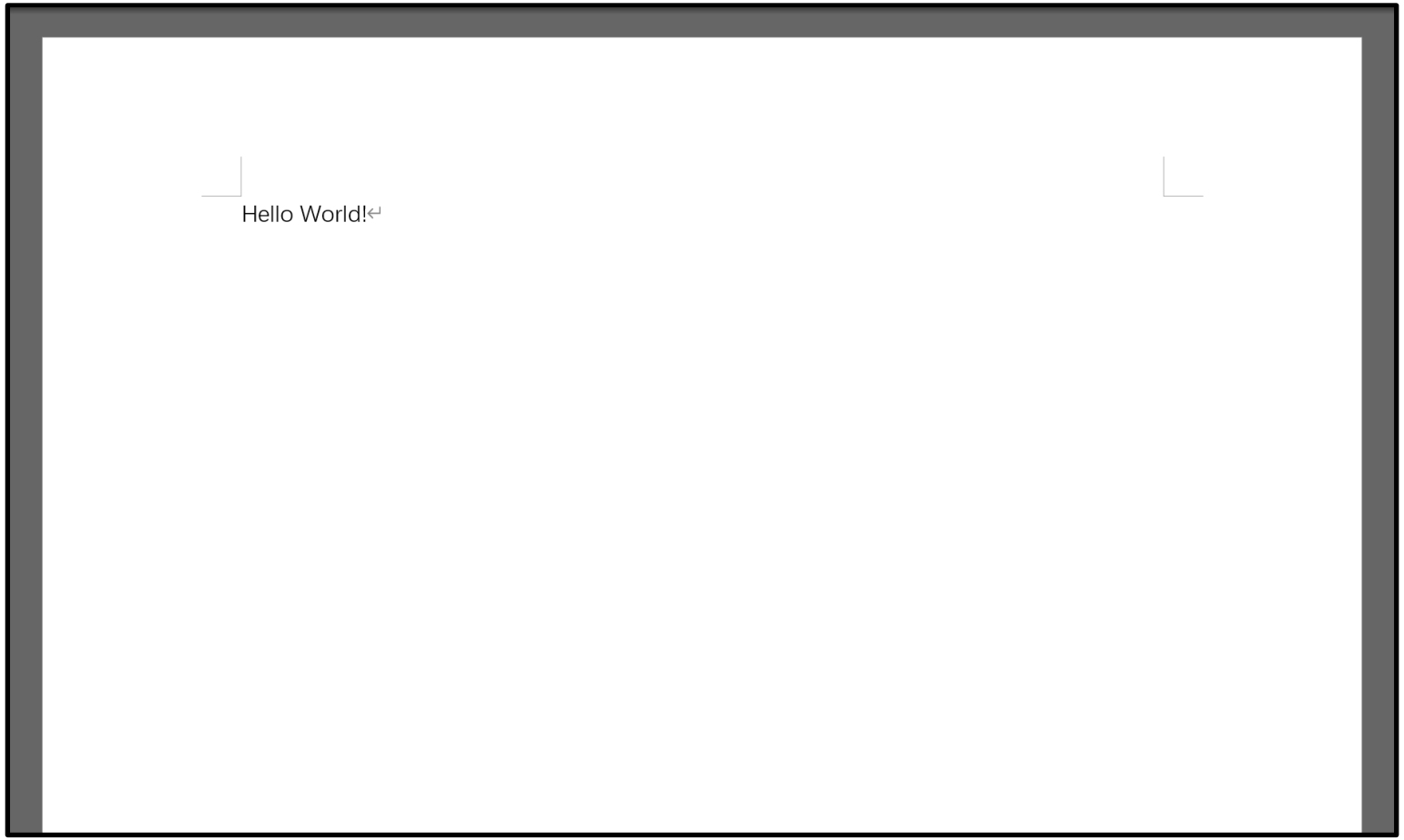
Visual Studio Guide

- Copy the result to the clipboard, then paste it into a Word file for printing. Try not to interrupt it unless an error message window pops up.
- Method: Select the content you want to copy, right-click the title bar of the result window, and select "Edit" → "Copy"



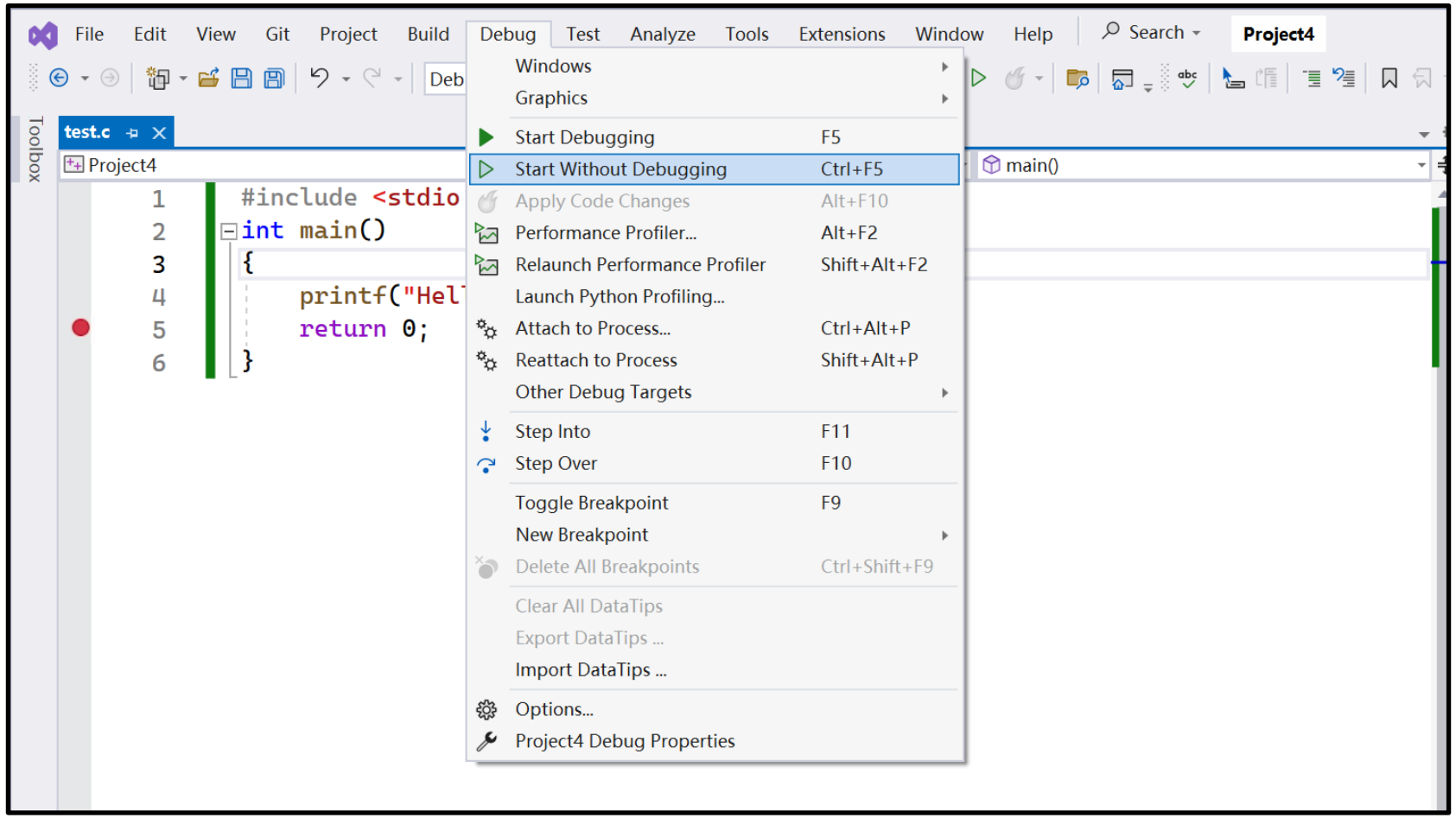
Visual Studio Guide

- **Paste the results into a Word document. Avoid using screenshots except for error messages.**



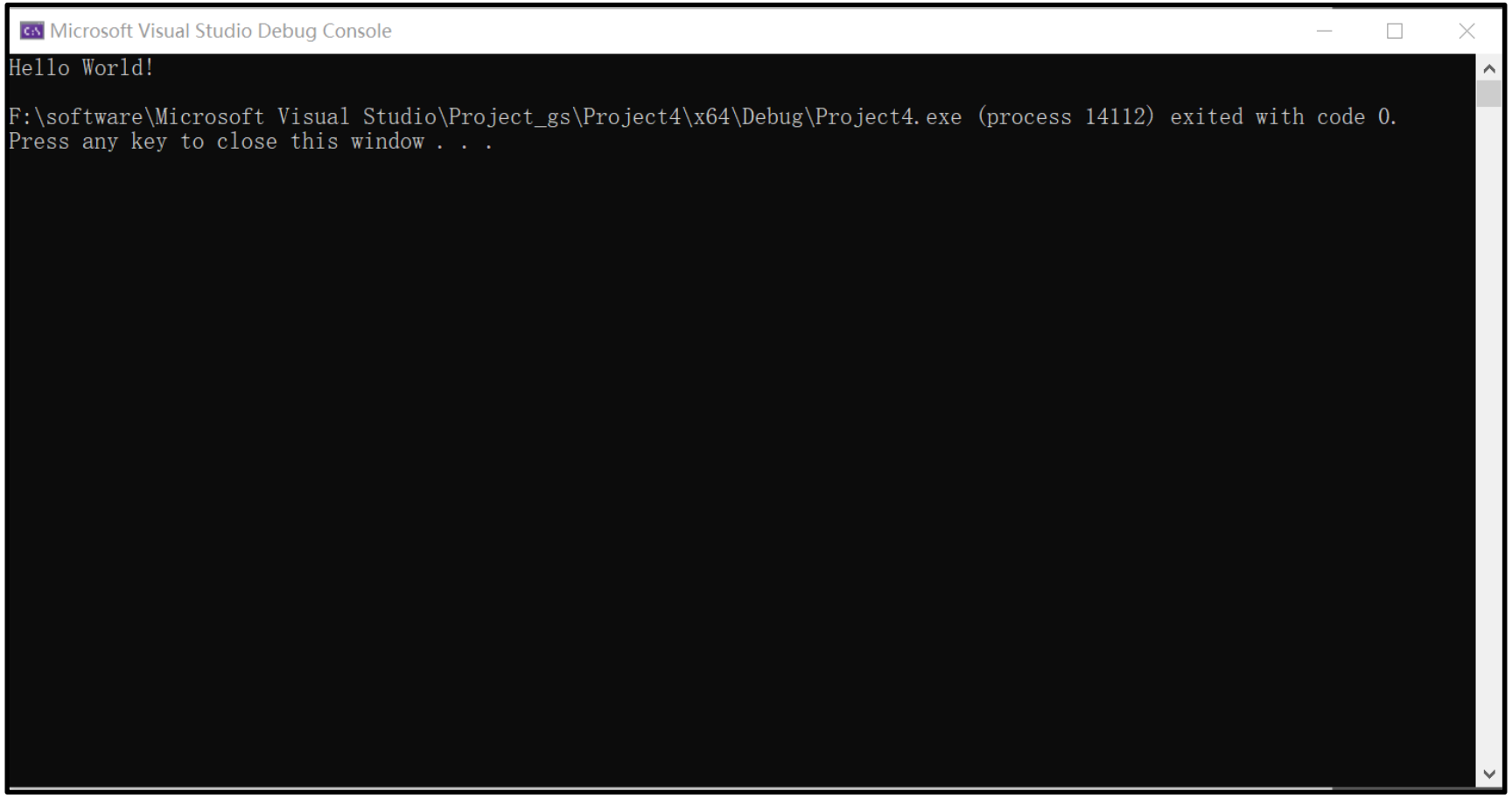
Visual Studio Guide

- **Executing the program: Select "Debug" -> "Start Without Debugging" or press Ctrl-F5.**



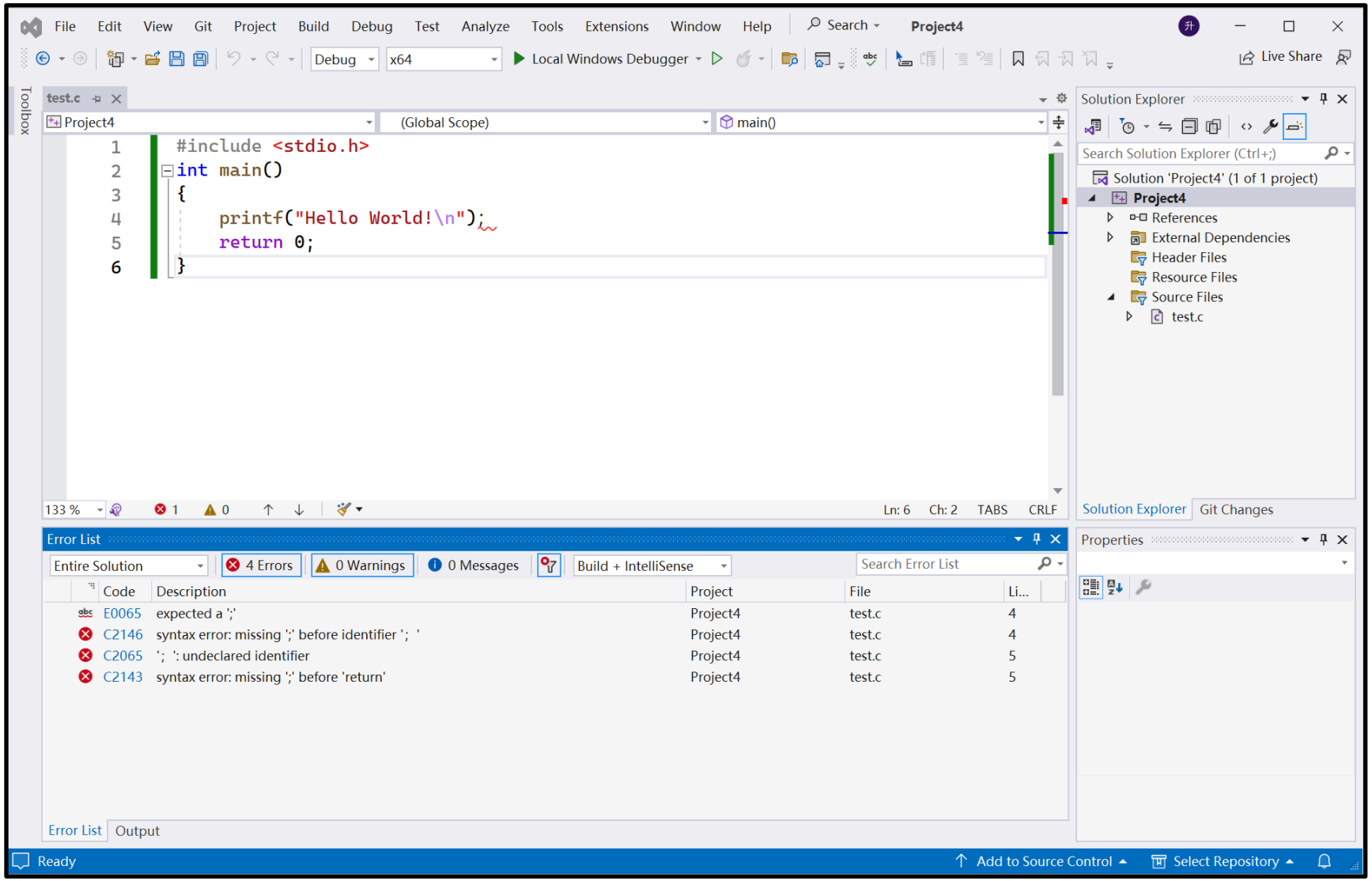
Visual Studio Guide

- **Obtaining results:**



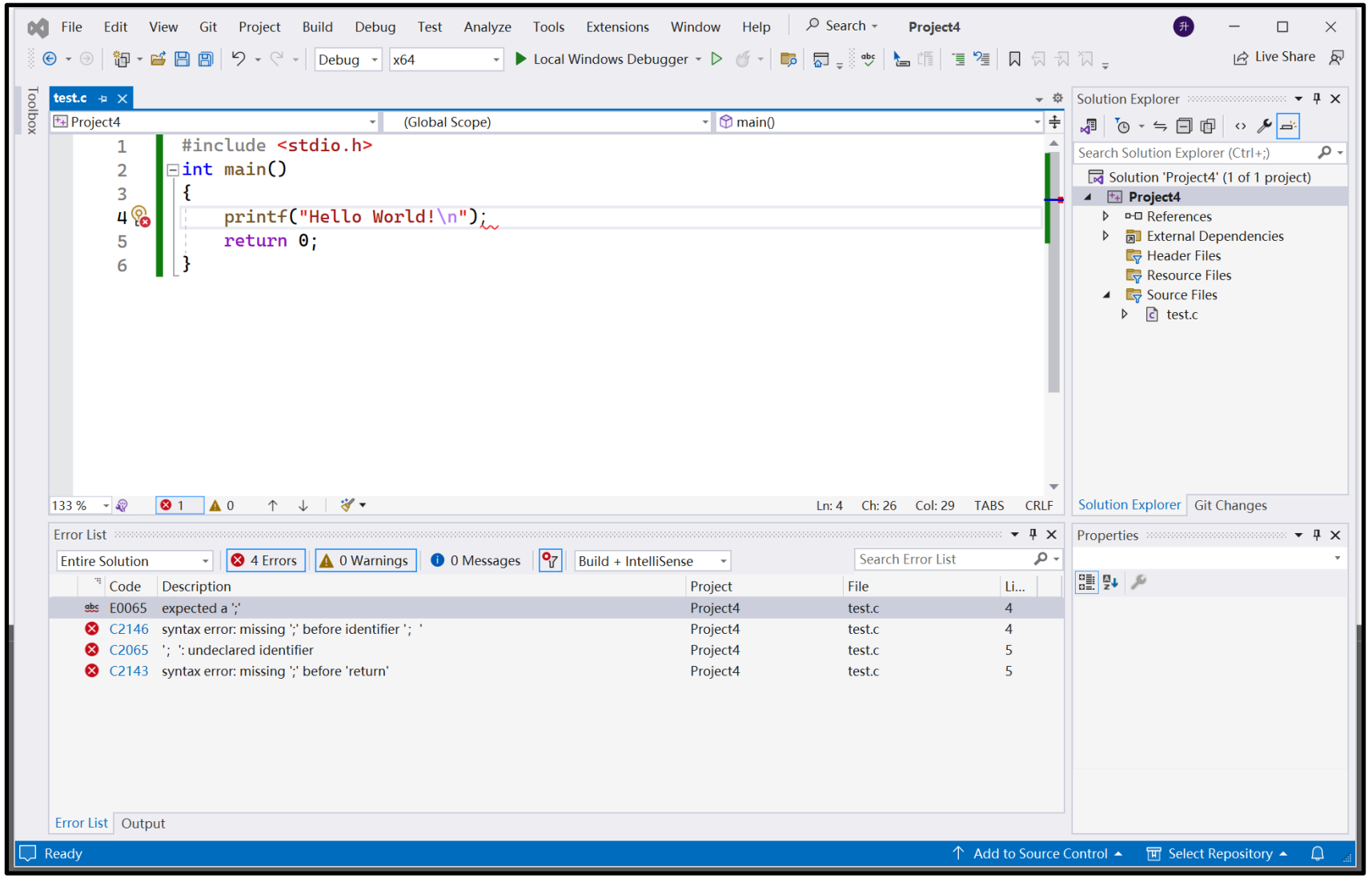
Visual Studio Guide

- **In case of compilation errors:**



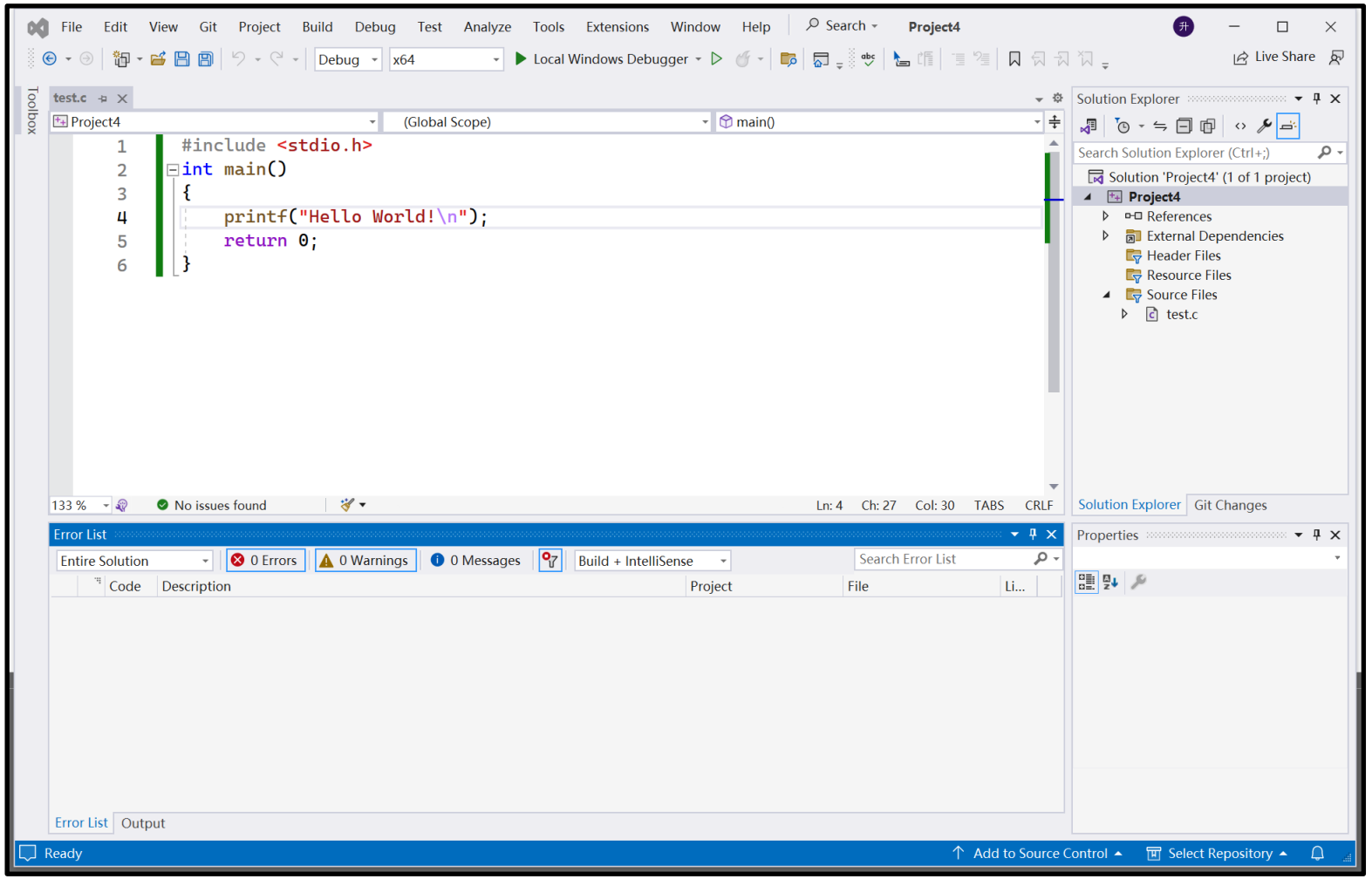
Visual Studio Guide

- **Double-click on the first error message to navigate to the problematic line.**



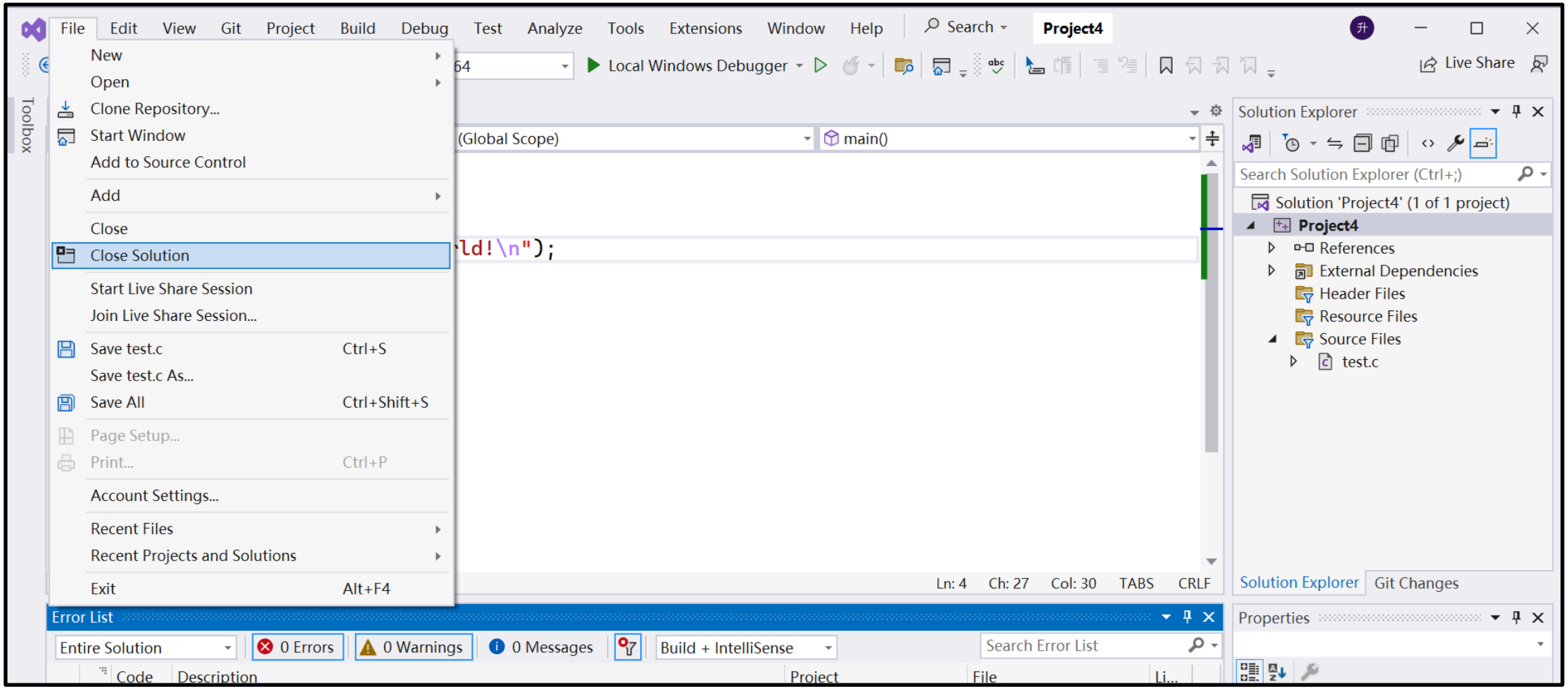
Visual Studio Guide

- **For example, if the error is due to a Chinese semicolon, replace it with an English semicolon and recompile.**



Visual Studio Guide

- **Closing and exiting Visual Studio.**



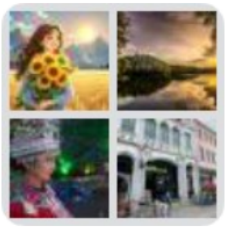
Homework

- **1st assignment: Book p.15, Exercises 1, 4, and 5**
  - **The homework of this time can be submitted by photographing handwritten work and uploading to the Tsinghua Web Learning.**
- **1st computer-based experiments: Simple Programming.**
  - **Book p.15, Exercises 6, and 7 (no lab report required).**
  - **Note: (1) The square root function is: double sqrt(double).**

    (2) Include #include <math.h> at the beginning of your program.

    (3) To read a double variable a from keyboard, use scanf_s("%lf", &a);.

Class WeChat Group



群聊：2025-2026 C Language

该二维码7天内(9月26日前)有效，重新进入将更新